Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

DIPLOMOVÁ PRÁCE

Simulátor konečných automatů

2026

Bc. Jiří Mlčoušek

Vedoucí práce:
doc. RNDr. Miroslav Kolařík, Ph.D.

Studijní program: Aplikovaná informatika,
Specializace: Vývoj software

Bibliografické údaje

Autor:	Bc. Jiří Mlčoušek
Název práce:	Simulátor konečných automatů
Typ práce:	diplomová práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Aplikovaná informatika, Specializace: Vývoj software
Vedoucí práce:	doc. RNDr. Miroslav Kolařík, Ph.D.
Počet stran:	68
Přílohy:	elektronická data v úložišti katedry informatiky
Jazyk práce:	český

Bibliographic info

Author:	Bc. Jiří Mlčoušek
Title:	Finite automaton simulator
Thesis type:	master thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Applied Computer Science, Specialization: Software Development
Supervisor:	doc. RNDr. Miroslav Kolařík, Ph.D.
Page count:	68
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Czech

Anotace

Tato diplomová práce se zabývá návrhem a implementací interaktivního simulátoru a vizualizátoru modelů z teorie formálních jazyků. Teoretická část práce podrobně mapuje Chomského hierarchii s důrazem na konečné automaty, zásobníkové automaty. Práce dále popisuje klíčové algoritmy pro transformaci a minimalizaci těchto modelů, které tvoří jádro realizované aplikace určené pro podporu výuky teoretické informatiky.

Synopsis

This master thesis focuses on the design and implementation of an interactive simulator and visualizer for models in formal language theory. The theoretical part provides a detailed map of the Chomsky hierarchy, emphasizing finite automata, pushdown automata, and Turing machines. Furthermore, the thesis describes key algorithms for transformation and minimization of these models, which form the core of the implemented application designed to support the education of theoretical computer science.

Klíčová slova: formální jazyky; konečné automaty; vizualizace; simulace; algoritmy; Chomského hierarchie

Keywords: formal languages; finite automata; visualization; simulation; algorithms; Chomsky hierarchy

Rád bych poděkoval vedoucímu práce doc. RNDr. Miroslavu Kolaříkovi, Ph.D. za cenné rady, trpělivost a odborné vedení během zpracování této diplomové práce. Dále děkuji své rodině a blízkým za podporu při studiu.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1 Úvod **10**
1.1 Kontext a motivace . . . 10
1.2 Problematika a potřeba řešení . . . 10

2 Teoretický základ **11**
2.1 Formální gramatiky a jazyky . . . 11
2.2 Chomského hierarchie . . . 11
2.3 Konečné automaty a regulární jazyky . . . 12
2.3.1 Deterministický konečný automat (Deterministický konečný automat (DKA)) . . . 13
2.3.2 Nedeterministický konečný automat (Nedeterministický konečný automat (NKA)) . . . 13
2.3.3 Konečný automat s ε-kroky (Nedeterministický konečný automat s epsilon přechodama (ε-NKA)) . . . 14
2.4 Regulární výrazy . . . 14
2.4.1 Převod regulárního výrazu na konečný automat . . . 15
2.5 Operace nad konečnými automaty . . . 16
2.5.1 Převod ε-NKA na NKA . . . 16
2.5.2 Převod NKA na DKA (Determinizace) . . . 16
2.5.3 Minimalizace konečných automatů . . . 16
2.6 Zásobníkové automaty a bezkontextové jazyky . . . 17
2.6.1 Princip fungování zásobníku . . . 18
2.6.2 Akceptační módy zásobníkového automatu . . . 18
2.6.3 Odlišná role nedeterminismu a deterministické automaty . 18
2.6.4 Nedeterministický zásobníkový automat . . . 19
2.6.4.1 Akceptační módy Nedeterministický zásobníkový automat (NZA) a ekvivalence . . . 20
2.6.4.2 Výpočetní síla NZA oproti Deterministický zásobníkový automat (DZA) . . . 20

3 Návrh systému **20**
3.1 Analýza požadavků a cílová skupina . . . 20
3.1.1 Funkční a nefunkční požadavky . . . 21
3.2 Hierarchická analýza úloh (HTA) . . . 22
3.3 Prototypování a návrh UI . . . 22
3.3.1 Wireframy (Nízká věrnost) . . . 22
3.3.2 High Fidelity (Hi-Fi) Mockup (Vysoká věrnost) . . . 23

4 Implementace **24**
4.1 Použité technologie . . . 24
4.1.1 C# a .NET 10 . . . 24
4.1.2 ASP.NET Core 10 . . . 25
4.1.3 ASP.NET Core Identity . . . 25

4.1.4 Entity Framework Core 10 (EF Core) 25
4.1.5 Microsoft SQL Server 26
4.1.6 Front-endové technologie 26
4.1.6.1 HTML5 26
4.1.6.2 CSS3 & Bootstrap 5 27
4.1.6.3 JavaScript (ES6) 27
4.1.7 Kontejnerizace a orchestrace 27
4.1.7.1 Docker a Docker Compose 27
4.1.7.2 .NET Aspire 28
4.1.8 Doplňkové technologie a knihovny 28
4.1.8.1 xUnit 28
4.1.8.2 Testcontainers 28
4.1.8.3 Shouldly 29
4.1.8.4 OpenTelemetry 29
4.1.8.5 Cytoscape.js, Dagre a Cytoscape-Dagre 29
4.1.9 Vývojové nástroje a prostředí 29
4.1.9.1 Visual Studio 2026 29
4.1.9.2 Docker Desktop 30
4.1.9.3 GitHub 30
4.1.9.4 GitHub Copilot 30
4.2 Softwarová architektura a struktura projektů 30
4.2.1 Dekompozice aplikačního jádra (Layered Architecture) . . 31
4.2.2 Ověřování kvality (Testovací blok) 32
4.2.3 Kontejnerizace a Orchestrace (Nasazovací blok) 32
4.2.4 Aplikace principů SOLID 32
4.2.5 Návrhové vzory a správa závislostí 33
4.2.5.1 Inverze řízení a Dependency Injection (Dependecy Injection (DI)) 33
4.2.5.2 Vzor Strategie (Strategy Pattern) 34
4.2.5.3 Vzor Dekorátor (Decorator Pattern) 35
4.2.6 Životní cyklus požadavku a Observabilita 35
4.2.6.1 Vlastní Middleware a Action filtry 35
4.2.6.2 Tři pilíře observability 36
4.3 Implementace matematických algoritmů 36
4.3.1 Minimalizace deterministického konečného automatu . . . 36
4.3.1.1 Fáze algoritmu 36
4.3.2 Převod NFA na DFA (Podmnožinová konstrukce) 41
4.3.2.1 Průběh algoritmu 41
4.3.2.2 Složitost 41
4.3.3 Eliminace ε-přechodů (ε-NKA na NKA) 42
4.3.3.1 ε-uzávěr 42
4.3.3.2 Transformace automatu 42
4.3.4 Thompsonova konstrukce (regulární výraz $\rightarrow$ ε-NFA) . . . 43
4.3.4.1 Fáze 1: Tokenizace s podporou znakových tříd . . 43

4.3.4.2 Fáze 2: Infix → Postfix . . . 44
4.3.4.3 Fáze 3: Fragmentová konstrukce ε-NFA . . . 44
4.3.5 Generování náhodných automatů . . . 44
4.3.5.1 Strategie generování . . . 45
4.3.6 Generování vstupních řetězců . . . 46
4.3.6.1 Generování přijímaného vstupu (BFS) . . . 46
4.3.6.2 Náhodná procházka (Random Walk) . . . 47
4.3.6.3 Generování zamítaného vstupu . . . 47
4.3.6.4 Souhrnná tabulka složitostí . . . 47
4.4 Vizualizační systém a interaktivní plátno . . . 48
4.4.1 Architektura vizualizačního systému . . . 48
4.4.1.1 Klíčové komponenty . . . 48
4.4.2 Interakce s backendem . . . 53

5 Zajištění kvality a testování 54
5.1 Význam testování v moderním vývoji softwaru . . . 54
5.1.1 Testy jako vývojářská pojistka . . . 54
5.2 Pyramida testů a testovací strategie . . . 55
5.2.1 Jednotkové testy (Unit Tests) – Základ pyramidy . . . 55
5.2.2 Integrační testy (Integration Tests) – Střední vrstva . . . 56
5.2.3 End-to-End testy (E2E) – Vrchol pyramidy . . . 57
5.3 Závislosti jako zdroj komplikací . . . 57
5.4 Vlastnosti kvalitních testů . . . 57
5.5 Kontinuální integrace a automatizace . . . 58
5.6 Role testů při práci s umělou inteligencí . . . 58
5.7 Závěr kapitoly . . . 58

6 Uživatelská dokumentace 59

7 Testování a experimenty 59

8 Potenciální rozšíření a pokračování 59
8.1 Funkční rozšíření . . . 59
8.1.1 Turingovy stroje . . . 59
8.1.2 Nedeterministické zásobníkové automaty . . . 59
8.1.3 Formální gramatiky . . . 59
8.1.4 Ekvivalence automatů . . . 59
8.2 Technická vylepšení . . . 60
8.2.1 Progressive Web App (PWA) . . . 60
8.2.2 Mobilní responsivita . . . 60
8.2.3 Collaborative Real-Time Editing . . . 60
8.3 AI asistence . . . 60
8.3.1 Návrh automatu pomocí přirozeného jazyka . . . 60
8.4 Výzkumný potenciál . . . 60

Závěr 61

Conclusions 62

A Návrh uživatelského rozhraní (HCI prezentace) 63

B Testování při vývoji softwaru (Integrační testy pomocí testcontainers) 63

C Obsah elektronických dat 63

Seznam zkratek 65

Literatura 66

Seznam tabulek

1 Časové a prostorové složitosti implementovaných algoritmů. Symboly: $|Q|$ – počet stavů, $|\Sigma|$ – velikost abecedy, $|T|$ – počet přechodů, L – maximální délka slova, k – počet pokusů, $|r|$ – délka reg. výrazu, n, m – cílový počet stavů/přechodů. 48

Seznam vět

1 Definice (Formální gramatika) 11
2 Definice (Deterministický konečný automat) 13
3 Definice (Nedeterministický konečný automat) 13
4 Definice (Nedeterministický konečný automat s ε-přechody) 14
5 Definice (Regulární výraz) 14
6 Definice (Zásobníkový automat) 17
7 Definice (Deterministický zásobníkový automat) 19
8 Definice (Nedeterministický zásobníkový automat) 19

1 Úvod

1.1 Kontext a motivace

Teorie formálních jazyků a automatů tvoří fundamentální teoretický základ moderní informatiky. Zkoumá samotnou podstatu algoritmické vyčíslitelnosti, poskytuje formální aparát pro analýzu složitosti algoritmů a jasně definuje limity toho, co lze pomocí výpočetních modelů řešit. Každý rozhodovací problém [1, kapitola 6.1] lze v tomto kontextu matematicky formulovat jako problém rozpoznávání příslušného formálního jazyka.[2, kapitola 1] [1, kapitola 6.2] Tento přístup umožňuje sjednotit pohled na nejrůznější informatické úlohy a poskytuje exaktní rámec pro jejich zkoumání.

Tuto komplexní problematiku systematicky klasifikuje tzv. Chomského hierarchie, [3] která rozděluje formální gramatiky a k nim příslušné jazyky do několika tříd s odlišnou výpočetní silou. Na vrcholu této hierarchie (typ 0) stojí obecné formální systémy reprezentované Turingovými stroji, zatímco směrem k nižším úrovním přecházíme k výpočetně slabším, avšak algoritmicky snáze analyzovatelným modelům. Ačkoliv se teoretická informatika může jevit jako primárně matematická disciplína, její praktická aplikace v softwarovém inženýrství je nepopiratelná, zejména při tvorbě překladačů, návrhu řídicích jednotek nebo při formální verifikaci systémů. V centru pozornosti této diplomové práce leží výpočetní modely spadající do dvou nejnižších úrovní Chomského hierarchie. Základní vrstvu tvoří deterministické (DKA), nedeterministické (NKA) a ε-nedeterministické konečné automaty (ε-NKA), jež slouží jako rozpoznávače regulárních jazyků (typ 3). Práce však rozšiřuje svůj obzor i o úroveň výše, k jazykům bezkontextovým (typ 2), a to začleněním teorie a implementace zásobníkových automatů (Zásobníkový konečný automat (ZKA)). Zatímco konečné automaty narážejí na limit daný absencí neomezené paměti, zásobníkové automaty tento problém překonávají obohacením o potenciálně nekonečnou datovou strukturu pracující na principu Last in first out (LIFO) – zásobník [5]. Zavedení tohoto mechanismu otevírá možnosti k řešení zcela nových, kvalitativně složitějších problémů, jako je kontrola správného uzávorkování či syntaktická analýza programovacích jazyků [1, kapitola 4.5].

1.2 Problematika a potřeba řešení

Přes svůj zásadní význam představuje výuka teorie automatů pro studenty informatiky dlouhodobou výzvu. Abstraktní matematické definice stavových přechodů, nedeterministické výpočty či mechanismy přechodových funkcí jsou obtížně uchopitelné bez konkrétní vizualizace a možnosti experimentování. Tradiční přístup založený na statických diagramech v učebnicích a ručním trasování výpočtů na papíře často vede k frustraci a povrchnímu porozumění probírané látky.

Současné softwarové nástroje pro výuku automatů, jako je široce používaný JFLAP [4], trpí zastaralým uživatelským rozhraním a technologickými omezeními. Jejich provoz vyžaduje instalaci specifického běhového prostředí, což kom-

plikuje nasazení. Webové alternativy pak často pokrývají úzké spektrum problematiky (obvykle jen DKA a NKA), postrádají transformační algoritmy či možnosti perzistence vytvořených modelů. Často také trpí zastaralým nebo uživatelsky nepřívětivým uživatelským rozhranním.

2 Teoretický základ

Text této kapitoly je systematicky strukturován tak, že postupuje od nejobecnějších konceptů rozhodovacích problémů a Chomského hierarchie až k detailnímu popisu specifik regulárních a bezkontextových jazyků a principů fungování jednotlivých automatů. Tento teoretický fundament následně slouží jako nezbytný odrazový můstek pro praktickou část, jež dokumentuje proces softwarového návrhu, tvorby přívětivého uživatelského rozhraní a samotnou implementaci webové aplikace.

2.1 Formální gramatiky a jazyky

V teorii formálních jazyků chápeme jazyk abstraktně jako množinu slov (řetězců) vytvořených ze symbolů určité konečné abecedy[6, kapitola 1.2]. Pro systematický popis a generování těchto jazyků slouží formální gramatiky.

Definice 1 (Formální gramatika)
Gramatika G je matematicky definována jako uspořádaná čtveřice $G = (N, \Sigma, P, S)$, kde:

- N je konečná neprázdná množina neterminálních symbolů (neterminálů).
- Σ je konečná množina terminálních symbolů (terminálů), pro kterou platí $N \cap \Sigma = \emptyset$.
- P je konečná množina přepisovacích pravidel.
- $S \in N$ je počáteční neterminál.

Aplikací přepisovacích pravidel se z počátečního neterminálu postupně odvozují slova, jež se skládají čistě z terminálních symbolů. Množina všech takto odvoditelných slov tvoří jazyk generovaný danou gramatikou $L(G)$ [7, kapitola 17].

2.2 Chomského hierarchie

V této kapitole jsou informace primárně čerpány na základě citovaného článku „Chomsky Hierarchy“ [3] z webu devopedia.org a dále z doporučené literatury.

Na konci 50. let 20. století lingvista Noam Chomsky klasifikoval formální gramatiky do čtyř základních tříd (typů) na základě restrikcí kladených na tvar jejich přepisovacích pravidel. Každý z těchto typů gramatik odpovídá specifické

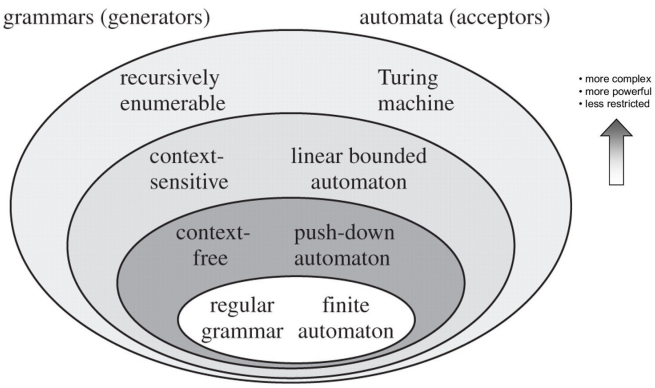
třídě jazyků a příslušnému výpočetnímu modelu, který dokáže tyto jazyky rozpoznávat:

1. **Gramatiky typu 0 (neomezené):** Na tvar pravidel nejsou kladena žádná omezení. Tyto gramatiky generují třídu rekurzivně spočetných jazyků, jež jsou rozpoznatelné univerzálním modelem výpočtu – Turingovými stroji.

2. **Gramatiky typu 1 (kontextové):** Pravidla umožňují přepisování neterminálů pouze v určitém kontextu a nesmí zkracovat délku přepisovaného řetězce (s možnou výjimkou pravidla $S \rightarrow \epsilon$). Těmto gramatikám odpovídají lineárně omezené automaty (LBA).

3. **Gramatiky typu 2 (bezkontextové):** Levá strana každého pravidla smí obsahovat pouze jeden izolovaný neterminál. Tyto gramatiky se široce využívají pro definici syntaxe programovacích jazyků a odpovídají jim ZKA.

4. **Gramatiky typu 3 (regulární):** Pravidla jsou silně omezena – neterminál se může přepsat pouze na jeden terminál, případně na terminál následovaný (nebo předcházený) jedním neterminálem. Tyto gramatiky generují regulární jazyky, jejichž rozpoznávačem je konečný automat.



Obrázek 1: Chomského hierarchie formálních gramatik a příslušných automatů.

Jak již bylo zmíněno tato práce se zabývá gramatikami typu 3 (regulárními) a gramatikami typu 2 (bezkontextovými).

2.3 Konečné automaty a regulární jazyky

Tato kapitola vychází z doporučené literatury a primárně z práce „Teoretická informatika– učební text“ od pana Prof. RNDr Petr Jančar, CSc. a konkrétně z kapitoly 3 [1, kapitola 3].

Základním výpočetním modelem, jenž se nachází na nejnižší úrovni Chomského hierarchie, je konečný automat. Jedná se o zařízení s konečnou vnitřní pamětí (ve formě stavů), které je schopno rozpoznávat regulární jazyky. Z hlediska návrhu se rozlišuje deterministický a nedeterministický konečný automat, přičemž obě varianty disponují totožnou výpočetní silou.

2.3.1 Deterministický konečný automat (DKA)

Definice 2 (Deterministický konečný automat)
DKA je formálně reprezentován pěticí $A = (Q, \Sigma, \delta, q_0, F)$, kde:

- Q je konečná množina stavů.
- Σ je vstupní abeceda.
- $\delta : Q \times \Sigma \to Q$ je přechodová funkce, která aktuálnímu stavu a přečtenému symbolu přiřadí jednoznačně stav následující.
- $q_0 \in Q$ je počáteční stav.
- $F \subseteq Q$ je množina koncových (akceptujících) stavů.

Stroj je deterministický, protože v každém kroku výpočtu existuje přesně jedna možnost, kam přejít. Výpočet je úspěšný (slovo je přijato do jazyka $L(A)$), pokud automat po přečtení celého slova skončí v některém ze stavů náležících do F.

2.3.2 Nedeterministický konečný automat (NKA)

Při návrhu algoritmů je často výhodné využít nedeterminismus.

Definice 3 (Nedeterministický konečný automat)
NKA je formálně reprezentován pěticí $A = (Q, \Sigma, \delta, q_0, F)$, kde:

- Q je konečná neprázdná množina stavů.
- Σ je konečná vstupní abeceda.
- $\delta : Q \times \Sigma \to 2^Q$ je přechodová funkce, která aktuálnímu stavu a přečtenému symbolu přiřadí množinu možných následujících stavů (kde 2^Q označuje potenční množinu množiny Q).
- $q_0 \in Q$ je počáteční stav.
- $F \subseteq Q$ je množina koncových (akceptujících) stavů.

Nedeterministický konečný automat se od DKA liší svou přechodovou funkcí. Ta je definována jako $\delta : Q \times \Sigma \to 2^Q$. To znamená, že pro jeden stav a jeden vstupní symbol může automat disponovat vícero možnostmi, do jakých stavů přejít. Slovo je automatem akceptováno, pokud existuje alespoň jeden možný výpočet (cesta stavovým grafem), který vede do koncového stavu. Nedeterminismus nezvyšuje výpočetní sílu (třídu rozpoznatelných jazyků), ale podstatně usnadňuje a zpřehledňuje návrh automatů.

2.3.3 Konečný automat s ε-kroky (ε-NKA)

Ještě vyšší míru abstrakce poskytuje automat s ϵ-přechody.

Definice 4 (Nedeterministický konečný automat s ε-přechody)
ε-NKA je formálně reprezentován pěticí $A = (Q, \Sigma, \delta, q_0, F)$, kde:

- Q je konečná neprázdná množina stavů.
- Σ je konečná vstupní abeceda.
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ je přechodová funkce, která aktuálnímu stavu a přečtenému symbolu, nebo prázdnému slovu ε (tzv. ε-krok), přiřadí množinu možných následujících stavů.
- $q_0 \in Q$ je počáteční stav.
- $F \subseteq Q$ je množina koncových (akceptujících) stavů.

Takový automat může změnit svůj vnitřní stav „spontánně“, tedy i bez přečtení symbolu ze vstupní pásky. Přechodová funkce je v tomto případě rozšířena na tvar $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q$. Tento přístup výrazně usnadňuje algoritmickou syntézu složitějších automatů, například při kompozici dvou automatů v operacích zřetězení nebo iterace jazyka.

2.4 Regulární výrazy

Zatímco konečné automaty slouží primárně jako rozpoznávače (akceptory) slov z určitého jazyka, regulární výrazy poskytují deklarativní způsob, jak takový jazyk formálně popsat [1, kapitola 3.12]. V softwarovém inženýrství a programátorské praxi nacházejí regulární výrazy široké uplatnění, například při vyhledávání textových vzorů v editorech, při definování vyhledávacích dotazů nebo při lexikální analýze v překladačích programovacích jazyků.

Z hlediska teoretické informatiky jsou regulární výrazy zcela ekvivalentní konečným automatům – oba tyto formalismy popisují přesně stejnou třídu, a to třídu regulárních jazyků [6].

Definice 5 (Regulární výraz)
Množina regulárních výrazů nad abecedou Σ, označovaná jako $RV(\Sigma)$, je definována induktivně [1, kapitola 3.12]:

- Symboly $\emptyset$, ε a symbol a (pro každé písmeno $a \in \Sigma$) jsou elementární regulární výrazy.
- Jestliže jsou α a β regulární výrazy, pak i zápisy $(\alpha + \beta)$, $(\alpha \cdot \beta)$ a (α^*) jsou regulární výrazy.

Každý regulární výraz α reprezentuje konkrétní regulární jazyk, který se značí jako $L(\alpha)$ (případně $[\alpha]$). Pro elementární výrazy platí triviální pravidla: $L(\emptyset) = \emptyset$, $L(\varepsilon) = \{\varepsilon\}$ a $L(a) = \{a\}$. Pro složenější výrazy odpovídají syntaktické operace přesně regulárním operacím nad jazyky:

- **Sjednocení:** $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$
- **Zřetězení:** $L(\alpha \cdot \beta) = L(\alpha) \cdot L(\beta)$
- **Iterace (Kleeneho hvězdička[1]):** $L(\alpha^*) = (L(\alpha))^*$.

2.4.1 Převod regulárního výrazu na konečný automat

Zásadním poznatkem pro praktickou implementaci nástrojů pracujících s regulárními výrazy je fakt, že ke každému regulárnímu výrazu lze algoritmicky sestrojit ekvivalentní konečný automat [1, kapitola 3.12].

V praxi (a tedy i v implementaci naší aplikace) se k tomuto účelu standardně využívá induktivní algoritmus známý jako **Thompsonova konstrukce**. Algoritmus postupně transformuje syntaktický strom regulárního výrazu na zobecněný nedeterministický konečný automat s ε-přechody (ε-NKA) [9].

Algoritmus garantuje, že výsledný ε-NKA má právě jeden počáteční stav, do kterého nevedou žádné hrany, a právě jeden koncový stav, ze kterého nevedou žádné hrany. Konstrukce probíhá podle následujících pravidel:

1. **Elementární výrazy:** Pro výrazy $\emptyset$, ε a $a \in \Sigma$ se zkonstruují triviální automaty obsahující dva stavy (počáteční a koncový) propojené příslušným přechodem.
2. **Sjednocení ($\alpha + \beta$):** Vytvoří se nový počáteční a koncový stav. Z nového počátečního stavu se vedou ε-přechody do původních počátečních stavů automatů pro α a β. Z jejich koncových stavů se následně svedou ε-přechody do nového koncového stavu.
3. **Zřetězení ($\alpha \cdot \beta$):** Koncový stav automatu pro α se plynule propojí s počátečním stavem automatu pro β pomocí ε-přechodu (nebo dojde k jejich přímému sloučení do jednoho stavu).
4. **Iterace (α^*):** Přidá se nový počáteční a koncový stav. Vytvoří se ε-přechody zajišťující samotnou iteraci (z koncového stavu α zpět do jeho počátečního), ε-přechod pro vynechání iterace (z nového počátečního rovnou do nového koncového stavu) a přechody pro napojení původního automatu.

Tímto elegantním postupem lze i složitý regulární výraz mechanicky transformovat do podoby grafu ε-NKA. Tento graf je následně pomocí algoritmů odstranění ε-kroků a determinizace (popsaných v následující podkapitole) převeden do deterministické podoby nezbytné pro efektivní softwarové vyhodnocování.

[1] Operátor je pojmenován po americkém matematikovi Stephenu C. Kleenem. Ten v roce 1956 jako první formálně definoval regulární výrazy a zavedl symbol hvězdičky pro operaci iterace při zkoumání konečných automatů a neuronových sítí [8].

2.5 Operace nad konečnými automaty

Pro efektivní práci s konečnými automaty v počítačových systémech je nutné ovládat řadu transformací. Každý model zprostředkovávající snazší návrh lze automatizovaně zredukovat do efektivní deterministické podoby.

2.5.1 Převod ε-NKA na NKA

Základním krokem při transformacích je eliminace ε-přechodů. Principem převodu je přidání nových přímých přechodů pro přečtení symbolu. Pro každý stav a vstupní symbol a se vyhledají všechny stavy, kterých lze dosáhnout libovolným počtem ε-kroků, následným čtením znaku a a opět libovolným počtem ε-kroků. Tyto nalezené cesty se následně nahradí standardním přechodem na symbol a. Po přidání takových hran lze z grafu smazat původní ε-hrany, aniž by došlo ke změně přijímaného jazyka. [7, kapitola 11]

2.5.2 Převod NKA na DKA (Determinizace)

Ke každému NKA lze najít ekvivalentní DKA pomocí tzv. podmnožinové konstrukce. Algoritmus simuluje všechny paralelní výpočty nedeterministického automatu současně. Novou množinou stavů Q' vznikajícího DKA je množina podmnožin původních stavů (odpovídající potenční množině 2^Q). Za počáteční stav nového automatu je brána množina obsahující počáteční stav (y) z NKA. V každém kroku se pro danou podmnožinu stavů a daný vstupní symbol a určí sjednocení všech stavů, do kterých se lze přesunout v původním NKA. Koncovým stavem determinizovaného automatu je pak každá podmnožina, která obsahuje alespoň jeden koncový stav původního NKA. Ačkoli tento proces může v nejhorším případě vést k exponenciálnímu nárůstu počtu stavů, v praxi se vytvářejí a reprezentují pouze stavy, jež jsou skutečně dosažitelné. [1, kapitola 3.9][7, kapitola 10]

2.5.3 Minimalizace konečných automatů

Vzhledem k tomu, že automaty slouží jako jádro reálných softwarových vyhledávačů, je klíčovým požadavkem jejich efektivita daná minimálním množstvím stavů. Jakýkoliv zadaný DKA lze převést na ekvivalentní minimální DKA.

Proces minimalizace se skládá ze dvou úkonů:

1. **Odstranění nedosažitelných stavů:** Pomocí prohledávání grafu do šířky od počátečního stavu se objeví všechny stavy, do kterých se automat může během čtení vstupu dostat. Stavy, které navštíveny nebyly, lze bezpečně odstranit.

2. **Ztotožnění ekvivalentních stavů (redukce):** Dva stavy p, q jsou považovány za ekvivalentní (jazykově nerozlišitelné), pokud automat zahajující výpočet jak z p, tak z q, obdrží na libovolném čteném retězci identický

výsledek (buď oba vstup přijmou, nebo oba zamítnou). Algoritmus postupně třídí stavy do rozkladových tříd. V nultém kroku se stavy rozdělí na přijímající a nepřijímající. Následně se třídy iterativně dělí na menší podskupiny (zjemňují), pokud existuje symbol, po jehož přečtení stavy z téže třídy přejdou do rozdílných rozkladových tříd. Když se již žádná skupina nerozpadá, proces končí. Z každé třídy ekvivalence je následně reprezentován pouze jeden stav a tím vznikne unikátní zredukovaný minimální automat.[1, kapitola 3.6][7, kapitola 9]

Pro zadaný deterministický konečný automat $M = (Q, \Sigma, \delta, q_0, F)$ algoritmus konstruuje minimální automat $M' = (Q', \Sigma, \delta', q_0', F')$, kde platí:

- $|Q'| \leq |Q|$ (výsledný automat má stejný nebo menší počet stavů),
- $L(M) = L(M')$ (oba automaty přijímají totožný jazyk),
- M' je minimální (neexistuje menší ekvivalentní DKA).

2.6 Zásobníkové automaty a bezkontextové jazyky

Zatímco konečné automaty poskytují vysoce efektivní rámec pro zpracování regulárních výrazů, nedokážou si poradit se složitějšími syntaktickými rysy, jakými je například párování nebo počítání neomezeného počtu struktur (např. otevíracích a zavíracích závorek) [1, kapitola 4]. Pro rozpoznávání bezkontextových jazyků (generovaných gramatikami typu 2) je k dispozici silnější abstraktní stroj: ZKA.

Definice 6 (Zásobníkový automat)
ZKA M je definován jako uspořádaná šestice $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$, kde:

- Q je konečná neprázdná množina stavů.
- Σ je konečná neprázdná množina vstupních symbolů (vstupní abeceda).
- Γ je konečná neprázdná množina zásobníkových symbolů (zásobníková abeceda).
- $q_0 \in Q$ je počáteční stav.
- $Z_0 \in \Gamma$ je počáteční zásobníkový symbol.
- δ je přechodová funkce, definovaná jako zobrazení množiny $Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma$ do množiny všech konečných podmnožin množiny $Q \times \Gamma^*$.

2.6.1 Princip fungování zásobníku

Zásobníkový automat si lze představit jako modifikaci konečného automatu obohacenou o dodatečnou potenciálně neomezenou datovou strukturu chovající se na principu LIFO zásobník [5]. Mechanika kroku tohoto stroje je závislá na aktuálním vnitřním stavu, načteném vstupním symbolu ze vstupní pásky (či ε-kroku) a symbolu umístěném na vrcholu zásobníku.

Zásobníkový automat pro každý výpočetní krok z vrcholu zásobníku aktuální prvek odebere a nahradí ho předepsanou sekvencí (která může být i prázdná, čímž dojde k odmazání vrchního symbolu). Přijetí slova se odehrává v závislosti na zvoleném akceptačním módu, jehož varianty jsou popsány v následující podkapitole.

2.6.2 Akceptační módy zásobníkového automatu

Zásobníkový automat může přijímat vstupní slova třemi způsoby, které se vzájemně liší podmínkami kladenými na stav automatu a obsah zásobníku po dočtení celého vstupního řetězce [1, kapitola 4]:

1. **Přijetí koncovým stavem:** Slovo w je přijato, pokud automat po dočtení w skončí v některém ze stavů $F \subseteq Q$ (množina přijímajících stavů). Obsah zásobníku v tomto okamžiku není zohledněn.

2. **Přijetí prázdným zásobníkem:** Slovo w je přijato, pokud automat po dočtení w kompletně vyprázdní zásobník (resp. zásobník obsahuje výhradně iniciální symbol dna). Vnitřní stav automatu v tomto okamžiku není zohledněn.

3. **Přijetí kombinací:** Slovo w je přijato, pokud jsou splněny obě předchozí podmínky současně – automat se nachází v přijímajícím stavu a zásobník je prázdný.

Pro **nedeterministické** zásobníkové automaty jsou všechny tři akceptační módy vzájemně ekvivalentní z hlediska vyjadřovací síly – ke každému ZKA přijímajícímu jedním způsobem lze sestrojit ekvivalentní ZKA přijímající libovolným jiným způsobem, přičemž všechny tři třídy přesně popisují třídu bezkontextových jazyků (CFL) [1, kapitola 4].

2.6.3 Odlišná role nedeterminismu a deterministické automaty

Zásadním rozdílem vůči prostým konečným automatům je role nedeterminismu. Zatímco u konečných automatů nemá nedeterminismus vliv na vyjadřovací sílu (každý NKA lze algoritmicky převést na ekvivalentní DKA pomocí podmnožinové konstrukce), ve třídě zásobníkových automatů je situace fundamentálně odlišná [1, kapitola 5.2].

Zásobníkové automaty s nedeterminismem dokážou akceptovat striktně větší množinu jazyků (bezkontextové jazyky, CFL) než jejich deterministické protějšky, které akceptují pouze tzv. deterministické bezkontextové jazyky (DCFL). Příkladem jazyka, který nedeterministický ZKA rozpozná, ale deterministický nikoliv, je jazyk palindromů – stroj nedokáže deterministicky uhodnout, kde se nachází střed slova a kdy má začít ze zásobníku znaky odebírat [6].

Abychom mohli o deterministickém chování hovořit formálně, zavedeme definici deterministického zásobníkového automatu (DZA).

Definice 7 (Deterministický zásobníkový automat)

Deterministický zásobníkový automat (zkráceně DZA) je zásobníkový automat $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$, pro jehož přechodovou funkci δ platí následující dvě omezující podmínky [1, kapitola 5.2]:

1. Pro každé $q \in Q$, $a \in \Sigma \cup \{\varepsilon\}$ a $X \in \Gamma$ obsahuje množina $\delta(q, a, X)$ nejvýše jeden prvek. (Pro daný stav, vstupní symbol a vrchol zásobníku existuje nanejvýš jedna instrukce.)

2. Pokud $\delta(q, \varepsilon, X) \neq \emptyset$, potom pro všechna $a \in \Sigma$ platí, že $\delta(q, a, X) = \emptyset$. (Pokud z dané konfigurace lze provést ε-krok, nesmí z ní existovat žádný krok čtoucí vstupní symbol.)

Tyto podmínky striktně zaručují, že v jakémkoliv okamžiku výpočtu existuje vždy nejvýše jedna aplikovatelná instrukce. Výpočet stroje je tedy lineární posloupností konfigurací a nedochází k žádnému větvení.

Na rozdíl od nedeterministického případu nejsou pro DZA všechny tři akceptační módy ekvivalentní. Lze ukázat, že třída DCFL přijímaná DZA koncovým stavem je striktně větší než třída jazyků přijímaná DZA prázdným zásobníkem [1, kapitola 5.2]. Prakticky nejvýznamnější variantou je přijetí koncovým stavem, neboť deterministické zásobníkové automaty tvoří teoretický základ efektivních deterministických syntaktických analyzátorů používaných při kompilaci moderních programovacích jazyků.

2.6.4 Nedeterministický zásobníkový automat

Nedeterministický zásobníkový automat (NZA) zobecňuje DZA tím, že uvolňuje podmínku jednoznačnosti přechodové funkce. Z jedné konfigurace tak může vést více různých kroků a automat zároveň „sleduje“ všechny možné větve výpočtu souběžně. Slovo je přijato, existuje-li *alespoň jedna* větev vedoucí do přijímající konfigurace.

Definice 8 (Nedeterministický zásobníkový automat)

Nedeterministický zásobníkový automat (NZA) M je uspořádaná šestice $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ se stejnými složkami jako DZA, avšak přechodová funkce je zobecněna na

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \longrightarrow 2^{Q \times \Gamma^*},$$

tj. každé trojici (stav, symbol nebo ε, vrchol zásobníku) přiřadí *konečnou množinu* dvojic (nový stav, řetězec nahrazující vrchol zásobníku). Na přechodovou funkci nejsou kladena žádná omezení determinismu [1, kapitola 4].

Definice konfigurace a relace $\vdash_M$ jsou totožné jako u DZA; pro NZA je pouze jeden krok $(q, aw, X\gamma) \vdash_M (q', w, \beta\gamma)$ podmíněn tím, že $(q', \beta) \in \delta(q, a, X)$.

2.6.4.1 Akceptační módy NZA a ekvivalence

NZA podporuje tytéž tři akceptační módy jako DZA. Na rozdíl od deterministického případu jsou však pro NZA všechny tři módy vzájemně ekvivalentní z hlediska vyjadřovací síly – ke každému NZA přijímajícímu jedním způsobem lze algoritmicky sestrojit ekvivalentní NZA přijímající libovolným jiným způsobem. Všechny tři třídy přesně popisují třídu bezkontextových jazyků (CFL) [1, kapitola 4].

2.6.4.2 Výpočetní síla NZA oproti DZA

Zásobníkové automaty s nedeterminismem dokážou akceptovat striktně větší množinu jazyků než jejich deterministické protějšky:

$$\text{DCFL} \subsetneq \text{CFL}.$$

Příkladem jazyka, který NZA rozpozná, ale DZA nikoliv, je jazyk palindromů – stroj nedokáže deterministicky uhodnout, kde se nachází střed slova a kdy má přejít ze zásobníkové fáze ukládání do fáze odebírání [6]. Tato skutečnost zásadně odlišuje zásobníkové automaty od konečných automatů, kde determinismus a nedeterminismus výpočetní sílu neovlivňují.

3 Návrh systému

Návrh uživatelského rozhraní a celkové architektury systému byl realizován s důrazem na pravidla přístupnosti a použitelnosti. Celý proces návrhu, od úvodní analýzy přes HTA až po tvorbu prototypů, byl zpracován v rámci předmětu Human computer interaction (HCI)[2] pod vedením RNDr. Zdeňka Eichlera[3].

Hlavním problémem, který aplikace řeší, je potřeba jednoduchého, dostupného a interaktivního nástroje pro tvorbu, úpravu a testování konečných automatů. Cílem je poskytnout efektivní a edukativní grafické rozhraní, které studentům usnadní pochopení teoretických konceptů popsaných v předchozí kapitole.

3.1 Analýza požadavků a cílová skupina

Před samotným návrhem bylo nutné definovat, komu je aplikace určena a v jakém prostředí bude primárně využívána.

[2]Detailní informace a sylabus předmětu jsou dostupné v informačním systému STAG.
[3]Kompletní prezentace mapující tento proces je součástí příloh této práce.

Cílovou skupinu lze rozdělit do tří hlavních kategorií:

- **Studenti informatiky (začátečníci):** Primární uživatelé, kteří potřebují intuitivní prostředí pro vizualizaci algoritmů a testování svých řešení (např. domácích úkolů).
- **Učitelé a lektoři:** Uživatelé využívající aplikaci pro demonstraci učiva přímo ve výuce nebo při přípravě studijních materiálů.
- **Vývojáři a vědci:** Pokročilí uživatelé vyžadující nástroj pro rychlé ověření svých softwarových návrhů.

Aplikace je koncipována jako webová, s primárním zacílením na monitory osobních počítačů a notebooků (referenční rozlišení 1920×1080). Jelikož je předpokládáno nasazení ve školním prostředí, je kladen velký důraz na pravidla přístupnosti, dostatečný kontrast a čitelnost prvků.

3.1.1 Funkční a nefunkční požadavky

Na základě definice cílové skupiny byly stanoveny následující požadavky na systém:

Funkční požadavky:

- Tvorba a editace automatů (přidávání/mazání stavů a přechodů).
- Možnost zadání automatu třemi způsoby: manuálním klikáním v Grafické uživatelské rozhraní (GUI), textovým zápisem a importem ze souboru.
- Testování vstupních řetězců (okamžité spuštění i krokovací edukační režim).
- Podpora více typů automatů (DKA, NKA, ε-NKA, DZA) a jejich potencionální vzájemné převody.
- Minimalizace deterministických automatů.
- Generování automatů přímo ze zadaného regulárního výrazu.
- Export, sdílení a správa uložených projektů vázaná na uživatelský účet.

Nefunkční požadavky:

- Intuitivní, přehledné a responzivní GUI.
- Rychlá odezva systému na uživatelské akce probíhající na plátně.
- Přístupnost a vizuální jasnost (zejména zřetelné oddělení stavů, zvýraznění aktivních prvků při krokování).

3.2 HTA

Pro správné rozvržení uživatelského rozhraní byly definovány hlavní případy užití (Use-cases) a následně rozpracovány pomocí HTA. Ta dekomponuje složité činnosti na elementární kroky, které uživatel musí provést.

Mezi stěžejní analyzované úlohy patří například **Vytvoření nového automatu**, které uživateli nabízí dvě hlavní cesty – buď vizuální tvorbu klikáním na plátno (přidání stavu, hrany, zadání symbolu), nebo zápis pomocí textové definice, jež se okamžitě překresluje do grafu.

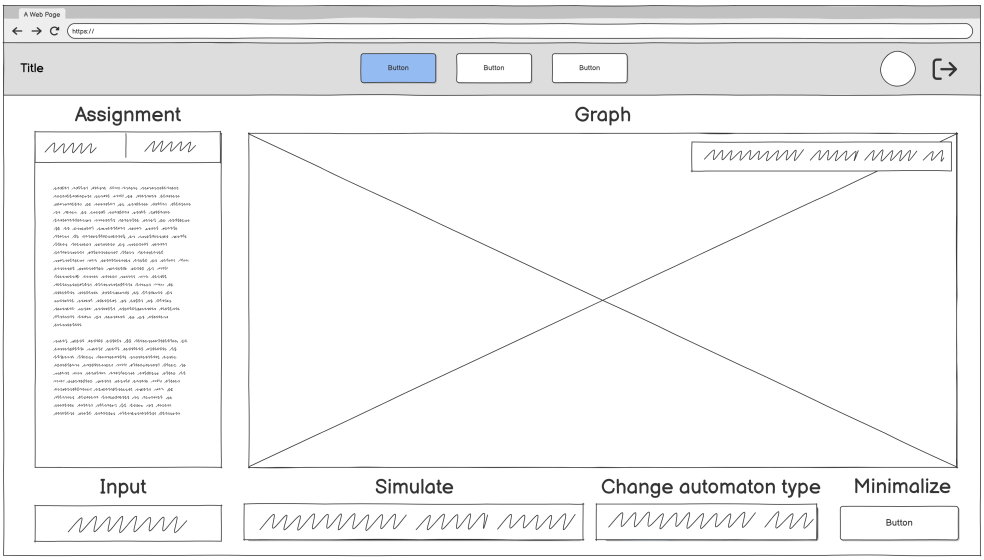
Další klíčovou úlohou je **Testování vstupního řetězce**, kde uživatel vloží zkoumané slovo. V případě volby krokování systém vizuálně zvýrazní aktuální stav a uživatel explicitně kliká na tlačítko pro provedení dalšího kroku výpočtu, čímž může detailně sledovat průchod grafem. Systém také pokrývá úlohy pro spouštění algoritmů (např. minimalizace či determinizace), které po vyvolání akce generují a zobrazují zcela nový stavový graf.

3.3 Prototypování a návrh UI

Na základě definovaných úloh (HTA) přešel návrh do fáze prototypování. Cílem bylo sjednotit pracovní plochu tak, aby uživatel měl neustále k dispozici jak textovou/strukturální definici automatu, tak jeho grafickou reprezentaci a ovládací prvky simulace.

3.3.1 Wireframy (Nízká věrnost)

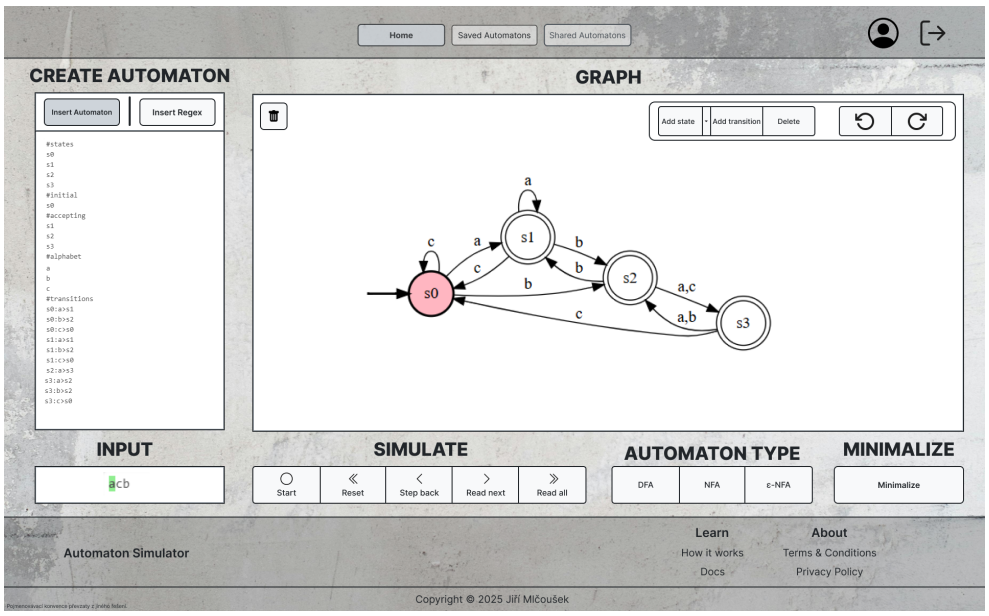
V první iteraci vznikly wireframy (náčrty), které rozvrhly obrazovku do logických bloků. Levý panel byl vyhrazen pro textové definice a zadávání vstupů, zatímco dominantní pravá část patřila interaktivnímu plátnu s grafem. Pod grafem byl umístěn ovládací panel pro spouštění simulace (ovládání podobné přehrávači médií – start, krok vpřed, krok vzad) a akční tlačítka pro transformace.



Obrázek 2: Prvotní návrh rozložení prvků (Wireframe).

3.3.2 Hi-Fi Mockup (Vysoká věrnost)

Následně proběhla evaluace wireframů a jejich převedení do podoby Hi-Fi prototypů, které již definovaly konkrétní barevnost, typografii a přesné tvary ovládacích prvků. Vizuální styl byl volen s ohledem na čistotu a minimalismus, aby pozornost uživatele směřovala výhradně na strukturu zpracovávaného automatu.



Obrázek 3: Finální návrh uživatelského rozhraní (Hi-Fi Mockup).

Tento iterativní proces od definice problému přes analýzu úloh až po vizuální prototypy zajistil, že výsledná implementace systému bude odpovídat potřebám

cílové skupiny a poskytne vysoce funkční edukační prostředí.

4 Implementace

Tato kapitola popisuje proces transformace navrženého řešení do plně funkční softwarové aplikace. Jsou zde rozepsány nejen zvolené technologické nástroje, ale také architektonické postupy a konkrétní implementační detaily stěžejních částí systému.

4.1 Použité technologie

Softwarové řešení bylo vybudováno na moderním technologickém stacku. Jádro celého systému je záměrně postaveno na úzce provázaném ekosystému technologií od společnosti Microsoft[4]. Volba tohoto technologického předního dodavatele jako hlavního poskytovatele softwarové infrastruktury garantuje nejen dlouhodobou podporu, vysokou stabilitu a pokročilé zabezpečení, ale především naprosto bezproblémovou vzájemnou integraci jednotlivých systémových komponent. Níže je uveden detailní výčet a popis těch nejzásadnějších technologií tvořících jádro aplikace.

4.1.1 C# a .NET 10

Jako hlavní programovací jazyk celého řešení byl zvolen C# ve verzi 14 běžící na nejnovější platformě .NET 10 [11]. Samotný vývoj aplikace byl původně zahájen nad platformou .NET 9 (s využitím tehdejší verze jazyka C#13), avšak díky striktnímu dodržování principů čisté architektury (Clean Architecture) a vysoké míře modularity bylo možné v průběhu vývojového cyklu provést plynulou a bezproblémovou migraci celého řešení na tyto aktuální technologie. Tato migrace přinesla řadu výkonnostních optimalizací a přístup k modernějším jazykovým konstruktům.

C# je moderní, silně typovaný a objektově orientovaný programovací jazyk vyvinutý společností Microsoft. Poskytuje striktní statickou typovou kontrolu, která je naprosto nezbytná pro spolehlivý vývoj [10]. Jazyk C# je klíčovým prvkem platformy .NET, ve které je tato webová aplikace vyvíjena. V rámci logické vrstvy byly hojně využity jeho pokročilé vlastnosti, jako je deklarativní dotazování pomocí Language Integrated Query (LINQ) pro efektivní manipulaci s kolekcemi.

.NET je otevřená multiplatformní vývojářská platforma sloužící jako běhové prostředí pro aplikace napsané v C#. Verze 10 zajišťuje velmi vysoký výpočetní výkon díky modernímu Just In Time (JIT) překladači a optimalizovanému správci paměti (Garbage Collector). Platforma navíc disponuje rozsáhlou standardní knihovnou (*Base Class Library*), která výrazně usnadňuje implementaci

[4] https://www.microsoft.com/

složitých datových struktur. Její multiplatformní charakter navíc garantuje naprosto bezproblémový běh aplikace jak ve vývojovém prostředí operačního systému Windows, tak následně v produkčních linuxových kontejnerech [11].

4.1.2 ASP.NET Core 10

Vysoce výkonný, open-source a cross-platform framework vyvinutý společností Microsoft pro budování moderních cloudových a webových aplikací. V rámci předloženého projektu plní roli hlavního aplikačního serveru a poskytuje kompletní zázemí pro zpracování HTTP požadavků [12]. ASP.NET Core využívá vysoce optimalizovaný webový server Kestrel a jeho architektura je založena na modulární *pipeline* tvořené takzvanými *middleware* komponentami. To umožňuje přesné řízení životního cyklu každého požadavku, počínaje jeho přijetím, přes zabezpečení, až po samotné směrování (routing) na příslušné controllery.

Framework rovněž nativně obsahuje robustní systém pro *Dependency Injection* (vstřikování závislostí), který byl v aplikaci využit pro předávání instancí služeb (například generátorů automatů či konvertorů) do jednotlivých vrstev aplikace. Tím je zajištěna nízká provázanost kódu (loose coupling) a vysoká testovatelnost řešení. ASP.NET Core se v tomto projektu stará nejen o dynamické generování HTML obsahu a obsluhu aplikačních rozhraní, ale také o rychlé a efektivní servírování statických souborů, jako jsou klientské skripty v jazyce JavaScript a kaskádové styly (CSS), jež obstarávají interaktivní vrstvu samotného simulátoru.

4.1.3 ASP.NET Core Identity

Knihovna nativně zabudovaná v ekosystému ASP.NET Core, jež poskytuje kompletní a vysoce zabezpečený *framework* pro autentizaci a autorizaci uživatelů. V kontextu budované aplikace řeší Identity veškerou správu uživatelských relací (sessions), bezpečné ukládání hesel pomocí moderních kryptografických hašovacích algoritmů a validaci uživatelských vstupů při registraci či přihlašování [13].

Klíčovým rysem, který byl v projektu využit, je podpora pro řízení přístupu na základě rolí (Role-Based Access Control). Díky tomu mohla být do aplikace snadno implementována logika pro sdílení automatů mezi více uživateli s různými úrovněmi oprávnění (vlastník, spolupracovník, pozorovatel). ASP.NET Core Identity navíc nevyžaduje psaní vlastních SQL dotazů pro správu uživatelů, jelikož je přímo napojeno na databázovou vrstvu přes Entity Framework Core.

4.1.4 Entity Framework Core 10 (EF Core)

Entity Framework Core je rozšiřitelný a multiplatformní ORM (Object-Relational Mapper) nástroj, který tvoří pomyslný most mezi aplikační logikou a samotnou

databází. Umožňuje programátorům pracovat s databázovými záznamy čistě pomocí objektově orientovaného přístupu. Namísto psaní surových SQL skriptů jsou data dotazována a manipulována prostřednictvím C# dotazů LINQ, což radikálně snižuje chybovost a zvyšuje bezpečnost proti útokům typu SQL Injection [14].

V projektu je striktně využíván přístup *Code-First*. To znamená, že databázové schéma není vytvářeno ručně, ale je automaticky generováno a aktualizováno pomocí takzvaných migrací na základě C# modelů (např. tříd reprezentujících uživatele nebo uložené automaty). EF Core zde slouží nejen pro ukládání aplikačních dat, ale funguje i jako perzistentní vrstva pro výše zmíněný systém ASP.NET Core Identity.

4.1.5 Microsoft SQL Server

Spolehlivý relační databázový systém podnikové třídy (RDBMS) od společnosti Microsoft, sloužící v této aplikaci jako primární trvalé úložiště dat. Uchovává veškeré informace o klientských účtech, přístupových právech a serializovaných strukturách vytvořených konečných automatů [15].

Díky plné kompatibilitě s Entity Framework Core dokáže SQL Server efektivně překládat abstraktní LINQ dotazy na vysoce optimalizované databázové operace.

4.1.6 Front-endové technologie

Zatímco logické jádro a aplikační server spoléhají výhradně na technologie společnosti Microsoft, klientská část aplikace (prezentační vrstva) je vybudována na univerzálních a otevřených webových standardech. Tyto technologie jsou nedílnou součástí vývoje jakékoliv moderní webové aplikace. Framework ASP.NET Core je pro spolupráci s těmito standardy nativně optimalizován – stará se o efektivní kompilaci šablon (Razor) do HTML struktury a rychlé servírování statických souborů (skriptů a stylů) přímo do prohlížeče uživatele. Pro realizaci uživatelského rozhraní byly využity následující technologie:

4.1.6.1 HTML5

Základní sémantický značkovací jazyk (HyperText Markup Language) v jeho nejnovější páté generaci, který definuje samotnou strukturu a obsah webových stránek. V kontextu této diplomové práce má HTML5 jeden naprosto kritický a nenahraditelný úkol: poskytuje element `<canvas>`. Tento prvek zprostředkovává aplikační rozhraní (Canvas 2D API) pro vysoce výkonné a nízkoúrovňové vykreslování grafiky [17]. Namísto přetěžování objektového modelu dokumentu (DOM) stovkami HTML uzlů pro každý stav a přechod automatu, umožňuje `<canvas>` plynulé a hardwarově akcelerované vykreslování celého grafu na úroveň jednotlivých pixelů.

4.1.6.2 CSS3 & Bootstrap 5

Kaskádové styly (Cascading Style Sheets) ve verzi 3 slouží k oddělení vizuální prezentace od logické struktury dokumentu. Pro zefektivnění vývoje, zajištění moderního vzhledu a responzivity napříč různými velikostmi obrazovek byl surový CSS kód doplněn o populární open-source framework Bootstrap ve verzi 5 [18], [19]. Tento framework poskytuje rozsáhlou sadu předpřipravených komponent (navigační lišty, modální okna, formulářové prvky, responzivní mřížku), díky čemuž bylo možné soustředit vývojářské úsilí primárně na složitou logiku simulátoru, nikoliv na základní stylování běžných uživatelských prvků.

4.1.6.3 JavaScript (ES6)

Univerzální skriptovací jazyk, který běží přímo v prohlížeči uživatele a vdechuje statické HTML struktuře život. Vzhledem k vysokým požadavkům na interaktivitu vizualizátoru nese JavaScript obrovskou část zodpovědnosti za uživatelský zážitek.

Aby byl kód dlouhodobě udržitelný a přehledný, nevyužívá aplikace žádné těžkotonážní frameworky (jako např. React či Angular), ale spoléhá na čistý JavaScript strukturovaný pomocí moderních *ES6 Modules* [20]. Toto paradigma umožňuje rozdělit složitou klientskou logiku do izolovaných a znovupoužitelných souborů. Skripty se tak starají o kompletní řízení chování vizualizačního plátna, složité přepočítávání souřadnic při přesunu uzlů, dynamické asynchronní updaty uživatelského rozhraní bez nutnosti znovunačtení stránky (AJAX/Fetch) a validaci vstupů ještě před jejich odesláním na server.

4.1.7 Kontejnerizace a orchestrace

Pro zajištění snadného vývoje, testování a bezproblémového nasazení aplikace do produkčního prostředí byly využity moderní nástroje pro kontejnerizaci a orchestraci služeb.

4.1.7.1 Docker a Docker Compose

Docker je platforma pro kontejnerizaci aplikací. Umožňuje zabalit samotnou webovou aplikaci i s jejím kompletním běhovým prostředím, systémovými knihovnami a závislostmi do standardizovaného, přenositelného kontejneru [21]. Tím je efektivně eliminován častý problém odlišného chování softwaru na různých zařízeních a je garantováno naprosto identické chování na vývojářském stroji i na cílovém produkčním serveru.

Pro spouštění a distribuci celého systému je navíc využíván nástroj Docker Compose, který umožňuje v jednom textovém souboru definovat a koordinovat vícekontejnerové aplikace (v případě tohoto projektu jde o synchronizovaný běh kontejneru webové aplikace a kontejneru databázového serveru). Výsledná aplikace bude zveřejněna a distribuována jako hotový obraz (image) prostřednictvím veřejného registru Docker Hub. Případným zájemcům o nasazení simulátoru na

vlastní infrastrukturu tak bude k plnohodnotnému spuštění stačit pouze stáhnutí příslušného konfiguračního souboru a zadání jediného terminálového příkazu.

4.1.7.2 .NET Aspire

.NET Aspire je relativně nový, specializovaný cloud-native stack od společnosti Microsoft určený pro budování pozorovatelných (observable) a na produkci připravených distribuovaných aplikací [16]. V tomto projektu slouží primárně k pokročilé orchestraci a koordinaci běhu aplikačních kontejnerů a SQL databáze během lokálního vývoje a integračního testování. Aspire automaticky řeší propojování služeb a injektuje potřebné spojovací řetězce (connection strings) přímo do aplikace.

Ačkoliv mým primárním nástrojem pro produkční distribuci softwaru zůstává Docker a s technologií .NET Aspire mám zatím méně praktických zkušeností, rozhodl jsem se pro ni v projektu vytvořit plnou podporu. Hlavní motivací k prozkoumání a integraci této technologie byla velmi atraktivní myšlenka sjednocení vývojářského prostředí. Naprosto stěžejním a nejužitečnějším prvkem .NET Aspire je pro mě jeho vývojářský řídicí panel (dashboard). Ten v reálném čase poskytuje obrovské množství telemetrických dat, strukturované logy z obou kontejnerů a detailní metriky, což usnadňuje ladění a celkové porozumění tomu, co se v aplikaci reálně děje.

4.1.8 Doplňkové technologie a knihovny

Nad rámec hlavního technologického stacku byla v projektu využita řada specializovaných open-source knihoven. Tyto doplňkové technologie neslouží jako primární stavební kameny aplikační architektury, ale efektivně řeší specifické problémy v oblastech testování softwaru, monitorování jeho běhu a vizualizace složitých datových struktur.

4.1.8.1 xUnit

Moderní, flexibilní a široce komunitou podporovaný framework pro psaní a spouštění automatizovaných testů na platformě .NET [22]. V projektu byl využit jako hlavní nástroj pro jednotkové testování (Unit Testing). Pomocí xUnit je důkladně testována veškerá doménová logika aplikace, především pak správnost matematických algoritmů pro transformace, determinizaci a minimalizaci konečných automatů.

4.1.8.2 Testcontainers

Knihovna, která radikálně zjednodušuje integrační testování tím, že vývojářům umožňuje programově spouštět a spravovat izolované Docker kontejnery přímo z testovacího kódu [23]. V této diplomové práci jsou Testcontainers využívány pro dočasné spouštění plnohodnotné instance Microsoft SQL Serveru. Integrační testy tak nepracují s omezenou in-memory databází, ale testují chování aplikace proti reálnému databázovému enginu, který je po dokončení testovací sady

automaticky zahozen.

4.1.8.3 Shouldly

Shouldly je aserční knihovna pro platformu .NET, která se zaměřuje na čitelnost testů a poskytuje velmi srozumitelné chybové výpisy v případě, že test selže [24]. Byla nasazena v kombinaci s frameworkem xUnit. Umožňuje psát ověřovací podmínky plynulým (fluent) způsobem, což výrazně zvyšuje expresivitu a udržovatelnost testovacího kódu.

4.1.8.4 OpenTelemetry

Standardizovaný open-source framework pro observabilitu, který poskytuje nástroje k instrumentaci, generování a sběru telemetrických dat (metrik, logů a trasování) [25]. V aplikaci je OpenTelemetry využito pro auditování a měření výkonu výpočetních algoritmů. Pomocí návrhového vzoru *Decorator* jsou zaznamenávány časy běhu jednotlivých transformací a následně exportovány pro případnou analýzu.

4.1.8.5 Cytoscape.js, Dagre a Cytoscape-Dagre

Pro řešení problematiky grafové vizualizace na front-endu byla využita sada specializovaných JavaScriptových knihoven. **Cytoscape.js** (verze 3.23.0) slouží jako robustní engine pro analýzu a teoretickou reprezentaci grafových sítí [26]. K ní je napojen algoritmus **Dagre** (verze 0.8.5) [27] prostřednictvím adaptéru **Cytoscape-Dagre** (verze 2.5.0) [28], který zajišťuje automatické, hierarchické a vizuálně čisté rozvržení (layout) uzlů a orientovaných hran na ploše. Je důležité zdůraznit, že tyto knihovny poskytují pouze technologické jádro pro výpočet souřadnic. Samotná interaktivní logika – jako je ruční přesouvání stavů, dynamické tvoření přechodů tažením myši a zejména vizuální krokování simulace – byla postavena zcela od základu pomocí vlastního řešení pracujícího s elementem `<canvas>`.

4.1.9 Vývojové nástroje a prostředí

Kvalita a rychlost vývoje softwarového díla je úzce spjata s použitými vývojovými nástroji. Během realizace této diplomové práce bylo využíváno moderní a profesionální softwarové vybavení.

4.1.9.1 Visual Studio 2026

Jako hlavní integrované vývojové prostředí (IDE) pro psaní C# a webového kódu posloužilo Microsoft Visual Studio [29]. Vývoj projektu byl historicky zahájen ve verzi Visual Studio 2022, avšak bezprostředně po jejím vydání byl celý projekt plynule přemigrován na nejnovější Visual Studio 2026. Toto prostředí poskytuje pokročilé nástroje pro ladění (debugging), profilaci paměti a nativní integraci s orchestrátorem .NET Aspire, což dramaticky urychlilo vývojový proces.

4.1.9.2 Docker Desktop

Desktopová aplikace poskytující lokální běhové prostředí pro správu kontejnerů [30]. Nástroj byl nezbytný pro bezproblémový běh databázových kontejnerů, lokální testování finálních obrazů aplikace (images) před jejich distribucí na Docker Hub a pro zajištění chodu technologie .NET Aspire přímo na vývojářském počítači.

4.1.9.3 GitHub

Nejpoužívanější cloudová platforma pro hostování softwarových projektů a správu verzí využívající systém Git [31]. V rámci diplomové práce byl GitHub využit jako centrální repozitář pro bezpečné zálohování zdrojových kódů, sledování historie provedených změn (verzování) a případnou dokumentaci vývoje.

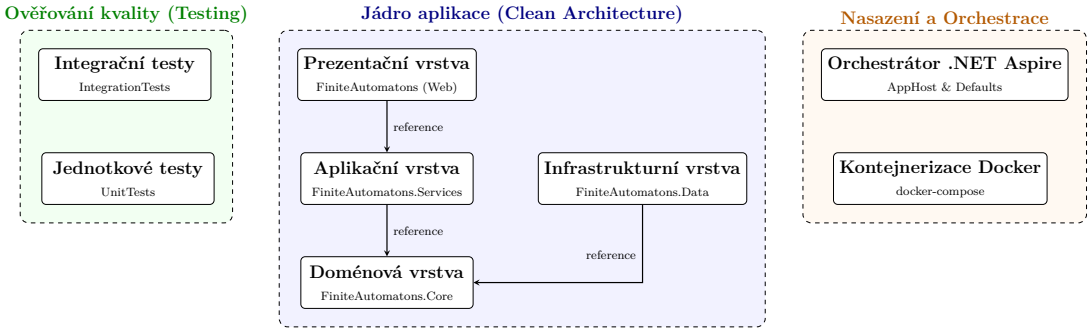
4.1.9.4 GitHub Copilot

Inovativní nástroj fungující jako programátorský asistent využívající modely umělé inteligence (AI) [32]. Během vývoje byl Copilot využit ve formě rozšíření do prostředí Visual Studio. Sloužil primárně ke zrychlení rutinních programátorských úkonů, generování struktur (boilerplate) pro testovací třídy a k asistenci při návrhu komplexnějších LINQ dotazů, čímž umožnil věnovat více času návrhu architektury a algoritmů.

4.2 Softwarová architektura a struktura projektů

Z důvodu zajištění dlouhodobé udržitelnosti, testovatelnosti a rozšiřitelnosti celého systému byla aplikace navržena v souladu s principy čisté architektury (*Clean Architecture*) a striktním oddělením zodpovědností (*Separation of Concerns*). Namísto tvorby monolitického bloku kódu je celkové řešení (Solution) dekomponováno do osmi oddělených projektů, které jsou logicky sdruženy do tří hlavních architektonických bloků: Jádro aplikace, Testování a Orchestrace.

Stěžejním pravidlem u jádra aplikace je tzv. pravidlo závislostí (Dependency Rule). Zdrojové kódy mohou odkazovat vždy pouze směrem dovnitř, tedy od infrastrukturních a prezentačních detailů směrem k čisté doménové logice. Vnitřní vrstvy nesmí mít žádnou povědomost o existenci vrstev vnějších. Samostatné bloky pro testování a nasazení následně toto jádro pouze zvenčí spravují či ověřují. Celý tento mechanismus je vizualizován na schématu 4.



Obrázek 4: Diagram závislostí projektů dekomponovaný do tří hlavních oblastí. V centru se nachází aplikační jádro striktně dodržující pravidlo směru závislostí. Po stranách jsou nezávislé bloky pro testování a orchestraci.

4.2.1 Dekompozice aplikačního jádra (Layered Architecture)

Projekty tvořící samotné jádro řešení jsou rozděleny do čtyř klíčových vrstev. Namísto přímého volání implementací se vrstvy propojují pomocí abstrakcí a DI kontejneru:

- **Doménová vrstva (*FiniteAutomatons.Core*):** Představuje absolutní středobod celého systému. Neobsahuje absolutně žádné závislosti na externích frameworcích (např. Entity Framework) nebo webovém prostředí. Nachází se zde čistá matematická abstrakce modelů z teorie formálních jazyků. Jednotlivé stavy a přechody jsou modelovány jako nezávislé datové struktury. Vrstva hojně využívá návrhový vzor Strategie ukrytý za rozhraním `IAutomaton`, což umožňuje jednotné zpracování deterministických (`DFA`), nedeterministických (`NFA`) i zobecněných (`EpsilonNFA`) automatů. Tato izolace garantuje, že matematická pravidla lze nezávisle testovat bez nutnosti spouštět webový server.

- **Aplikační vrstva (*FiniteAutomatons.Services*):** Tato vrstva plní roli hlavního orchestrátoru byznysových procesů. Definuje takzvané případy užití (Use-cases) a implementuje pro ně příslušná rozhraní (např. `IAutomatonExecutio` či `IAutomatonConversionService`). Jsou zde obsaženy komplexní algoritmy, jako je podmnožinová konstrukce nebo algoritmus pro minimalizaci. Návrh dodržuje paradigma *Fat Services* – veškerá výpočetní a transakční složitost leží zde, díky čemuž je kód snadno znovupoužitelný a připravený na případné nasazení i mimo webové prostředí (např. v konzolové aplikaci).

- **Infrastrukturní a datová vrstva (*FiniteAutomatons.Data*):** Vrstva primárně zodpovědná za persistenci dat a přístup k databázi. Skrývá složitost SQL dotazování za repozitářový vzor, který je nativně zajištěn nástrojem Entity Framework Core. Implementuje databázový kontext (`ApplicationDbConte`

definuje chování ORM mapování pomocí *Fluent API* a stará se o bezpečné aplikování Code-First migrací.

- **Prezentační vrstva (*FiniteAutomatons*):** Samotná webová aplikace spravující uživatelské HTTP požadavky. Využívá hybridní architekturu: pro zpracování operací nad automaty slouží návrhový vzor Model-View-Controller (MVC) s využitím tzv. *Thin Controllers* (tenkých řadičů), které veškerou práci pouze delegují na injektované služby z aplikační vrstvy. Oddělená architektura *Razor Pages* je naopak využita pro správu identity (přihlašování). Vrstva rovněž obsahuje vlastní *Middleware* komponenty (např. pro normalizaci symbolu ε z různých Unicode formátů) a řídí složitou obousměrnou synchronizaci mezi serverem a JavaScriptovým vizualizačním enginem Cytoscape.js.

4.2.2 Ověřování kvality (Testovací blok)

Vzhledem k teoretické a algoritmické povaze diplomové práce bylo nutné zajistit maximální spolehlivost implementovaných transformací. K tomuto účelu slouží na jádru nezávislý testovací blok tvořený dvěma projekty: *UnitTests* pro rychlé a izolované testování matematické logiky a *IntegrationTests* pro ověřování integrace komponent a celých webových scénářů (tzv. End-to-End) s využitím dočasně virtualizované databáze (*Testcontainers*). Detailní popis testovacích scénářů a metodiky je zevrubně zpracován v samostatné kapitole věnované testování.

4.2.3 Kontejnerizace a Orchestrace (Nasazovací blok)

Běh aplikace a její propojení s databází je definováno v rámci dvou odlišných, avšak koexistujících přístupů.

Hlavním a primárním nástrojem pro spouštění, distribuci a produkční nasazení celého systému je **Docker** (s využitím souboru *docker-compose.yml*). Tento přístup zaručuje, že si koncový uživatel nebo hodnotitel může aplikaci spustit na jakémkoliv zařízení jediným příkazem.

Jako alternativní podpora je do řešení integrována technologie **.NET Aspire** (zastoupená projekty *AppHost* a *ServiceDefaults*). Byla do projektu začleněna z důvodu vývojářské zvědavosti jakožto moderní prostředek pro sběr OpenTelemetry dat, poskytující přehledný interaktivní *dashboard* pro sledování logů a trasování běhu algoritmů přímo při vývoji. Architektura aplikace je však na tomto orchestrátoru absolutně nezávislá. Jeho případné odstranění z řešení by nemělo na produkční chod a funkčnost simulátoru sebemenší vliv.

4.2.4 Aplikace principů SOLID

Aby byla zajištěna maximální čistota kódu a jeho odolnost vůči budoucím změnám, je celá architektura simulátoru formována dle uznávaných návrhových principů SOLID [33]. Tyto principy definují standardy pro psaní flexibilního a snadno udržovatelného softwaru:

- **S – Single Responsibility Principle (Princip jedné odpovědnosti):** Každá třída v systému má pouze jediný důvod ke změně a plní právě jeden logický úkol. Příkladem je rozdělení aplikačních služeb: `AutomatonExecutionService` se stará exkluzivně o výpočet a krokování automatu, `AutomatonConversionService` řeší pouze matematické transformace a minimalizace, zatímco `SavedAutomatonServic` se věnuje výhradně perzistenci dat do databáze.

- **O – Open/Closed Principle (Princip otevřenosti/uzavřenosti):** Třídy by měly být otevřené pro rozšiřování, ale uzavřené pro modifikaci. Toho je v projektu dosaženo primárně využitím návrhového vzoru Dekorátor. Pokud bylo nutné přidat logování a audit k existujícím simulačním algoritmům, nebyl upravován jejich původní zdrojový kód. Namísto toho byla vytvořena nová obalovací třída, která původní funkcionalitu pouze rozšiřuje.

- **L – Liskov Substitution Principle (Liskovové princip substituce):** Odvozené třídy musí být schopny plně nahradit své bázové třídy bez narušení chodu programu. V doménovém modelu sdílí všechny typy automatů (`DFA`, `NFA`, `EpsilonNFA`) společné rozhraní `IAutomaton`. Služby a kontrolery pracují nad tímto obecným rozhraním a nezávisí na konkrétní implementaci, přičemž garantují, že zavolání metody `Execute()` vrátí korektní výsledek bez ohledu na to, o jaký typ vnitřního stavového stroje se jedná.

- **I – Interface Segregation Principle (Princip oddělení rozhraní):** Klienti by neměli být nuceni záviset na metodách, které nepoužívají. Místo tvorby jednoho masivního „božského rozhraní“ (*God Interface*) zastřešujícího veškerou logiku automatů, je systém rozdělen do striktně úzce zaměřených rozhraní (`IAutomatonExecutionService`, `IAutomatonConversionServic` atd.).

- **D – Dependency Inversion Principle (Princip inverze závislostí):** Vysokoúrovňové moduly nesmí záviset na nízkoúrovňových modulech; oba musí záviset na abstrakcích. MVC controllery (např. `AutomatonExecutionControlle` instanciují své závislosti výhradně pomocí rozhraní vkládaných skrze konstruktor (*Constructor Injection*). O dodání konkrétní implementace se stará DI kontejner spravovaný frameworkem ASP.NET Core.

4.2.5 Návrhové vzory a správa závislostí

K vyřešení specifických inženýrských problémů v rámci simulátoru byly na výše zmíněných principech vystavěny konkrétní návrhové vzory a techniky správy životního cyklu objektů.

4.2.5.1 Inverze řízení a Dependency Injection (DI)

Celá aplikace využívá mechanismus Inversion of Control (IoC) (Inversion of Cont-

rol). Komponenty si své závislosti nevytvářejí samy (např. klíčovým slovem `new`), ale jsou jim dodávány aplikačním serverem. Při spuštění aplikace (v souboru `Program.cs`) jsou všechny služby zaregistrovány do DI kontejneru s přesně určeným životním cyklem:

- **Transient (Dočasný):** Využito pro lehké výpočetní služby (např. generátory regulárních výrazů). Při každém vyžádání vzniká nová instance.
- **Scoped (Vázaný na HTTP požadavek):** Primární cyklus pro většinu služeb v projektu (např. `AutomatonExecutionService` a databázový `ApplicationDbContext`). Instance je vytvořena při přijetí HTTP požadavku z prohlížeče, sdílena napříč vrstvami během jeho zpracování a po odeslání odpovědi je bezpečně zničena, což garantuje izolaci dat mezi uživateli a vlákny.
- **Singleton (Jedináček):** Využito pro služby sdílené po celou dobu běhu serveru (např. globální konfigurace a poskytovatelé systémového času).

4.2.5.2 Vzor Strategie (Strategy Pattern)

Jelikož systém pracuje s deterministickými i nedeterministickými stroji, jejichž způsob výpočtu je diametrálně odlišný, využívá doménová vrstva vzor Strategie. Algoritmus výpočtu je zapouzdřen přímo v příslušné třídě:

```
public interface IAutomaton {
    bool Execute(string input);
    List<State> States { get; }
    List<Transition> Transitions { get; }
}

public class DFA : IAutomaton {
    public bool Execute(string input) {
        // Implementace primeho deterministickeho posunu
        State current = States.First(s => s.IsStart);
        foreach (char symbol in input) {
            var transition = Transitions.FirstOrDefault(t => t
                .FromStateId == current.Id && t.Symbol ==
                symbol);
            if (transition == null) return false;
            current = States.First(s => s.Id == transition.
                ToStateId);
        }
        return current.IsAccepting;
    }
}
```

Zdrojový kód 1: Aplikace vzoru Strategie v doménovém modelu automatu

4.2.5.3 Vzor Dekorátor (Decorator Pattern)

Návrhový vzor uplatněný k elegantnímu zajištění telemetrie. Třída dekorátoru implementuje stejné rozhraní jako výpočetní služba, obaluje ji a transparentně přidává logiku pro auditování.

```
public sealed class AutomatonExecutionServiceAuditorDecorator(
    AutomatonExecutionService inner, IAuditService audit) :
    IAutomatonExecutionService
{
        private readonly AutomatonExecutionService inner =
            inner ?? throw new ArgumentNullException(nameof(
            inner));
        private readonly IAuditService audit = audit ?? throw
            new ArgumentNullException(nameof(audit));

    public AutomatonViewModel ExecuteStepForward(
        AutomatonViewModel model) {
        // Zaznamenani auditu a nasledna delegace vypoctu na
            vnitrni sluzbu
        return MethodAuditor.AuditAsync(_audit, "
            ExecuteStepForward",
            () => Task.FromResult(_inner.ExecuteStepForward(
                model))
        ).GetAwaiter().GetResult();
    }
}
```

Zdrojový kód 2: Vzor Dekorátor aplikovaný pro infrastrukturu auditu

4.2.6 Životní cyklus požadavku a Observabilita

Proces zpracování dat (data flow) od uživatele až po samotný matematický výpočet vyžaduje v webovém prostředí sérii transformací.

4.2.6.1 Vlastní Middleware a Action filtry

Když uživatel z vizualizačního plátna odešle HTTP požadavek k provedení simulace, data narazí nejprve na *Middleware Pipeline* (sérii průtokových zpracovatelů). Pro potřeby simulátoru byl vyvinut vlastní `EpsilonNormalizationMiddleware`. Jelikož front-end (JavaScript, prohlížeč, operační systém uživatele) může posílat prázdné slovo v nejrůznějších formátech (znak ε, sekvence `\0`, prázdný řetězec, nebo zástupné znaky Unicode `\uFFFD`), tento middleware příchozí požadavek zachytí, analyzuje a normalizuje do jednotné systémové podoby.

Bezprostředně před vstupem struktury do MVC controlleru zasahuje `AutomatonModelFil` (tzv. Action Filter). Ten proaktivně zavolá metodu `EnsureInitialized()`, jež validuje a koriguje poškozené nebo neúplné kolekce stavů a hran, čímž chrání doménovou vrstvu před chybami prázdných referencí (NullReferenceException).

4.2.6.2 Tři pilíře observability

Moderní cloud-native aplikace vyžadují hluboký vhled do svého vnitřního fungování. Systém implementuje koncept takzvané observability založený na standardu *OpenTelemetry*, který stojí na třech pilířích:

1. **Trasování (Distributed Tracing):** Výpočetně náročné algoritmy (např. determinizace) otevírají vlastní `ActivitySource`. To umožňuje přesně měřit čas (milisekundy), jaký převod na serveru zabral, a detekovat případná výkonnostní hrdla s rostoucí velikostí grafu.

2. **Strukturované logování:** Aplikační události nejsou vypisovány jako prostý text, ale jako sémantické objekty (např. `{AutomatonType} s inputem {Input}`).

3. **Auditování:** Custom Audit Trail zaznamenává do perzistentního úložiště důležité byznysové operace (kdo, kdy a jaký automat spustil či uložil). V produkčním režimu jsou tyto záznamy směrovány do dedikovaných souborů pro případnou bezpečnostní analýzu.

4.3 Implementace matematických algoritmů

Jádro aplikace spočívá v korektní implementaci transformačních a analytických algoritmů známých z teoretické informatiky. Tato podkapitola rozebírá klíčové algoritmy zabudované do simulátoru, jejich matematické základy, praktickou realizaci v jazyce C# a komplexitní analýzu.

4.3.1 Minimalizace deterministického konečného automatu

Algoritmus minimalizace DKA implementuje přístup založený na zjemňování rozkladů. Cílem je ze zadaného DKA vytvořit ekvivalentní minimální automat, jenž rozpoznává stejný jazyk s minimálním možným počtem stavů.

4.3.1.1 Fáze algoritmu

Implementace se skládá z pěti oddělených fází:

Fáze 1: Analýza dosažitelnosti

Pomocí prohledávání grafu do šířky (Breadth-First Search) jsou identifikovány všechny stavy dosažitelné z počátečního stavu. Nedosažitelné stavy jsou následně z automatu odstraněny, jelikož nemohou ovlivnit přijímaný jazyk.

```
private (List<State> reachableStates, List<Transition>
    reachableTransitions)
     ComputeReachable(int startStateId)
{
     var reachable = new HashSet<int>();
     var queue = new Queue<int>();
     queue.Enqueue(startStateId);
     reachable.Add(startStateId);
```

```
8
9       while (queue.Count > 0)
10      {
11          var cur = queue.Dequeue();
12          foreach (var t in Transitions.Where(t => t.FromStateId
                == cur))
13          {
14              if (reachable.Add(t.ToStateId))
15                  queue.Enqueue(t.ToStateId);
16          }
17      }
18
19      var reachableStates = States.Where(s => reachable.Contains
            (s.Id)).ToList();
20      var reachableTransitions = Transitions
21          .Where(t => reachable.Contains(t.FromStateId) &&
22                      reachable.Contains(t.ToStateId)).ToList();
23      return (reachableStates, reachableTransitions);
24  }
```

Zdrojový kód 3: Výpočet dosažitelných stavů pomocí BFS

Časová složitost této fáze je $O(|Q| + |T|)$, kde $|T|$ označuje počet přechodů automatu.

Fáze 2: Inicializace rozkladu

Stavy jsou nejprve rozděleny do dvou základních tříd ekvivalence: množina koncových (přijímajících) stavů F a množina nekoncových stavů $Q \setminus F$. Tím vzniká počáteční rozklad.

```
1   private static List<HashSet<int>> CreateInitialPartitions(List
        <State> reachableStates)
2   {
3       var accepting = reachableStates
4           .Where(s => s.IsAccepting)
5           .Select(s => s.Id)
6           .ToHashSet();
7       var nonAccepting = reachableStates
8           .Where(s => !s.IsAccepting)
9           .Select(s => s.Id)
10          .ToHashSet();
11
12      var partitions = new List<HashSet<int>>();
13      if (accepting.Count > 0) partitions.Add([.. accepting]);
14      if (nonAccepting.Count > 0) partitions.Add([..
            nonAccepting]);
15      return partitions;
16  }
```

Zdrojový kód 4: Vytvoření iniciálního rozkladu

Fáze 3: Iterativní zjemňování rozkladu

Jádrem algoritmu je opakované dělení tříd ekvivalence na menší podtřídy.

```
private static List<HashSet<int>> RefinePartitions(
    List<HashSet<int>> partitions,
    List<Transition> transitions,
    List<char> symbols)
{
    bool changed;
    do
    {
        changed = false;
        var newPartitions = new List<HashSet<int>>();

        foreach (var group in partitions)
        {
            var splits = new Dictionary<string, HashSet<int
               >>();

            foreach (var stateId in group)
            {
                var signatureParts = new List<string>(symbols.
                   Count);
                foreach (var sym in symbols)
                {
                    var target = transitions.FirstOrDefault(t
                       =>
                        t.FromStateId == stateId && t.Symbol
                           == sym)?.ToStateId;
                    signatureParts.Add(target == null ? "null"
                        :
                        partitions.FindIndex(p => p.Contains(
                           target.Value))
                                .ToString());
                }

                var signature = string.Join("|",
                   signatureParts);
                if (!splits.TryGetValue(signature, out var
                   bucket))
                {
                    bucket = [];
                    splits[signature] = bucket;
                }
                bucket.Add(stateId);
            }

            if (splits.Count == 1)
```

```
            {
                newPartitions.Add([.. group]);
            }
            else
            {
                changed = true;
                foreach (var part in splits.Values)
                    newPartitions.Add([.. part]);
            }
        }

        partitions = newPartitions;
    } while (changed);

    return partitions;
}
```

Zdrojový kód 5: Iterativní zjemňování rozkladu

Korektnost a konvergence: Algoritmus iteruje, dokud se žádný rozklad dále nerozdělí. Jelikož při každé iteraci vznikají striktně jemnější rozklady a maximální počet tříd je omezen počtem stavů $|Q|$, algoritmus konverguje nejvýše za $O(|Q|)$ iterací. Časová složitost jedné iterace je $O(|Q|^2 \cdot |\Sigma|)$, celková složitost tedy činí $O(|Q|^3 \cdot |\Sigma|)$.

Poznámka: Stávající algoritmus je inspirován Hopcroftovým algoritmem. Původní Hopcroftův algoritmus dosahuje složitosti $O(|Q| \cdot |\Sigma| \cdot \log |Q|)$ použitím sofistikovanějších datových struktur. Předložená implementace klade důraz na čitelnost a srozumitelnost kódu.

Fáze 4: Konstrukce minimálního automatu

Z výsledných tříd ekvivalence se vytvoří nové stavy minimalizovaného automatu. Každá třída odpovídá právě jednomu stavu q' v minimálním automatu. Jako reprezentant je zvolen libovolný prvek třídy (všechny prvky jsou ekvivalentní). Přechody se konstruují na základě původních přechodů reprezentantů.

```
private static DFA BuildMinimizedDfa(List<HashSet<int>>
   partitions,
    List<State> reachableStates, List<Transition>
       reachableTransitions,
    int startStateId, List<char> symbols)
{
    var stateMap = new Dictionary<int, int>();
    var minimizedDfa = new DFA();
    int newId = 1;

    foreach (var group in partitions)
    {
        foreach (var oldId in group)
            stateMap[oldId] = newId;
```

```
13
14          var representativeId = group.First();
15          var repState = reachableStates.First(s => s.Id ==
               representativeId);
16
17          minimizedDfa.AddState(new State
18          {
19              Id = newId,
20              IsStart = group.Contains(startStateId),
21              IsAccepting = repState.IsAccepting
22          });
23          newId++;
24      }
25
26      foreach (var group in partitions)
27      {
28          var rep = group.First();
29          foreach (var sym in symbols)
30          {
31              var trans = reachableTransitions.FirstOrDefault(t
                   =>
32                  t.FromStateId == rep && t.Symbol == sym);
33              if (trans != null)
34              {
35                  int fromId = stateMap[rep];
36                  int toId = stateMap[trans.ToStateId];
37                  if (!minimizedDfa.Transitions.Any(t =>
38                      t.FromStateId == fromId && t.ToStateId ==
                           toId &&
39                      t.Symbol == sym))
40                      minimizedDfa.AddTransition(fromId, toId,
                           sym);
41              }
42          }
43      }
44
45      minimizedDfa.StateMapping = stateMap;
46      minimizedDfa.MergedStateGroups = stateMap
47          .GroupBy(kv => kv.Value)
48          .ToDictionary(g => g.Key, g => new HashSet<int>(g.
               Select(kv => kv.Key)));
49
50      return minimizedDfa;
51  }
```

Zdrojový kód 6: Konstrukce minimálního DFA

4.3.2 Převod NFA na DFA (Podmnožinová konstrukce)

Transformace nedeterministického konečného automatu na ekvivalentní deterministický automat je základní operací. Implementace využívá tzv. podmnožinové konstrukce [1].

Klíčová myšlenka: Každý stav výsledného DKA reprezentuje množinu stavů, ve kterých se NKA může současně nacházet.

4.3.2.1 Průběh algoritmu

Fáze 1 – Inicializace: Počáteční stav DKA tvoří jednoprvková množina obsahující počáteční stav NKA: $q_0^D = \{q_0^N\}$. Tento stav je přijímající právě tehdy, když $q_0^N \in F_N$.

Fáze 2 – Prohledávání stavového prostoru: Pomocí fronty (BFS) se systematicky procházejí všechny dosažitelné podmnožiny stavů NKA.

Fáze 3 – Výpočet přechodů: Pro danou podmnožinu stavů $S \subseteq Q_N$ a symbol $a \in \Sigma$ se definuje přechod:

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$$

```
private HashSet<int> ComputeNextStateSet(HashSet<int>
    currentSet, char symbol)
{
    var nextSet = new HashSet<int>();

    foreach (var nfaState in currentSet)
    {
        var transitions = Transitions.Where(t =>
            t.FromStateId == nfaState && t.Symbol == symbol);
        foreach (var transition in transitions)
        {
            nextSet.Add(transition.ToStateId);
        }
    }

    return nextSet;
}
```

Zdrojový kód 7: Výpočet následující množiny stavů

Nově objevené podmnožiny jsou přidány do fronty a zařazeny jako nové stavy DKA. Pro deduplikaci podmnožin se používá hašovací mapa s klíčem odvozeným od uspořádané sekvence identifikátorů stavů.

4.3.2.2 Složitost

V nejhorším případě může determinizace vést k exponenciálnímu nárůstu počtu

stavů: $|Q_D| = 2^{|Q_N|}$. Časová složitost pak činí $O(2^{|Q_N|} \cdot |\Sigma| \cdot |Q_N|)$. V praxi však většina automatů nedosahuje této horní meze a počet výsledných stavů bývá podstatně nižší, často blížící se $O(|Q_N|^2)$.

4.3.3 Eliminace ε-přechodů (ε-NKA na NKA)

Algoritmus odstraňuje ε-přechody pomocí výpočtu ε-uzávěrů stavů.

4.3.3.1 ε-uzávěr

Pro stav q definujeme ε-closure(q) jako množinu všech stavů dosažitelných z q pomocí nulového nebo více ε-přechodů. Implementace využívá prohledávání do hloubky (DFS).

```
private HashSet<int> EpsilonClosure(HashSet<int> states)
{
    var closure = new HashSet<int>(states);
    var stack = new Stack<int>(states);

    while (stack.Count > 0)
    {
        var state = stack.Pop();
        var epsilonTransitions = Transitions
            .Where(t => t.FromStateId == state && t.Symbol ==
               '\0');
        foreach (var transition in epsilonTransitions)
        {
            if (closure.Add(transition.ToStateId))
            {
                stack.Push(transition.ToStateId);
            }
        }
    }

    return closure;
}
```

Zdrojový kód 8: Výpočet epsilon-uzávěru

Složitost výpočtu jednoho uzávěru je $O(|Q| + |T_\varepsilon|)$, kde $|T_\varepsilon|$ označuje počet epsilon-přechodů.

4.3.3.2 Transformace automatu

Nová přechodová funkce NKA se definuje jako:

$$\delta_{NFA}(q, a) = \bigcup_{p \in \varepsilon\text{-closure}(q)} \varepsilon\text{-closure}(\delta_{\varepsilon NFA}(p, a))$$

Přijímající stav q v novém NKA je definován podmínkou:

$$q \in F_{NFA} \iff \varepsilon\text{-closure}(q) \cap F_{\varepsilon NFA} \neq \emptyset$$

Celková složitost transformace je $O(|Q|^2 \cdot |\Sigma| \cdot |T|)$.

4.3.4 Thompsonova konstrukce (regulární výraz → ε-NFA)

Pro převod regulárního výrazu na ε-NKA je implementován Thompsonův algoritmus[9], který rekurzivně skládá dílčí fragmenty automatu podle syntaktické struktury výrazu.

4.3.4.1 Fáze 1: Tokenizace s podporou znakových tříd

Lexikální analyzátor rozpoznává elementy regulárního výrazu včetně znakových tříd (např. `[a-z]`), které jsou expandovány do množin konkrétních symbolů.

```
private static (Tok Token, int EndIndex) ParseCharacterClass(
    string s, int startIndex)
{
    int j = startIndex + 1;
    var classChars = new List<char>();

    while (j < s.Length && s[j] != ']')
    {
        if (j + 2 < s.Length && s[j + 1] == '-' && s[j + 2] !=
            ']')
        {
            char start = s[j];
            char end = s[j + 2];
            if (start > end)
                throw new ArgumentException($"Neplatny rozsah
                    '{start}-{end}'");

            for (char cc = start; cc <= end; cc++)
                classChars.Add(cc);

            j += 3;
            continue;
        }

        classChars.Add(s[j]);
        j++;
    }

    var val = new string([.. classChars.Distinct()]);
    return (new Tok(TokType.CharClass, val), j);
}
```

Zdrojový kód 9: Parsování znakové třídy s rozsahy

Dále algoritmus explicitně vkládá operátory konkatenace mezi sousedící tokeny (např. `ab` → `a·b`).

4.3.4.2 Fáze 2: Infix → Postfix

Implementace Dijkstrova Shunting-Yard algoritmu transformuje regulární výraz z infixové notace do postfixové, která je vhodnější pro stromové vyhodnocení. Operátory mají definovanou precedenci: unární operátory $*$, $+$, ? mají precedenci 5, konkatenace $\cdot$ precedenci 4 a alternace $|$ precedenci 3.

4.3.4.3 Fáze 3: Fragmentová konstrukce ε-NFA

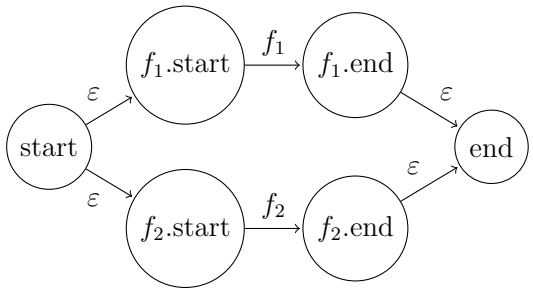
Algoritmus postupuje přes postfixovou reprezentaci výrazu a zásobníkově skládá dílčí fragmenty. Každý fragment má definovaný počáteční a koncový stav (exit stav ještě není označen jako přijímající).

Základní případy:

- **Znak** a: Fragment se dvěma stavy propojenými přechodem na symbol a.
- **Znaková třída** $[abc]$: Fragment se dvěma stavy, mezi nimiž vedou paralelní přechody pro každý symbol třídy.

Rekurzivní případy:

- **Konkatenace** $e_1 \cdot e_2$: Koncový stav fragmentu e_1 je propojen s počátečním stavem e_2 pomocí ε-přechodu.
- **Alternace** $e_1|e_2$: Vytvoří se nový počáteční stav s ε-přechody do obou fragmentů a nový koncový stav, do něhož vedou ε-přechody z obou původních koncových stavů.
- **Kleeneho hvězdička** e^*: Přidá se ε-přechod pro možnost vynechání (start → end) a ε-přechod pro iteraci ($\text{end}_e \to \text{start}_e$).



Obrázek 5: Struktura ε-NFA pro alternaci $e_1|e_2$ dle Thompsonovy konstrukce.

Celková časová i prostorová složitost Thompsonovy konstrukce je lineární: $O(|r|)$, kde $|r|$ je délka regulárního výrazu.

4.3.5 Generování náhodných automatů

Pro testovací a výukové účely implementuje aplikace generátor strukturovaně validních náhodných automatů s kontrolovanými parametry.

4.3.5.1 Strategie generování

Algoritmus pracuje ve třech fázích:

1. **Generování parametrů:** Náhodný výběr počtu stavů (5–15), počtu přechodů (4–25), velikosti abecedy (2–8) a podílu přijímajících stavů (20–50 %).

2. **Vytvoření stavů:** Stavy jsou číslovány od 1, první je označen jako počáteční. Přijímající stavy jsou vybrány náhodně.

3. **Generování přechodů s omezeními:**

   - *Dosažitelnost:* Vytvoří se spojující cesta $1 \to 2 \to \cdots \to n$, což garantuje, že všechny stavy jsou dosažitelné z počátečního stavu (slabá souvislost grafu).
   - *Pokrytí abecedy:* Algoritmus zajistí, že pro každý symbol $a \in \Sigma$ existuje alespoň jeden přechod označený symbolem a.
   - *Determinismus (pro DFA):* Kontroluje se, zda pro daný stav a symbol již přechod existuje. Pokud ano, je náhodně vybraný kandidát zamítnut.

```
private static int EnsureConnectivity(
    AutomatonType type,
    List<State> states,
    List<char> alphabet,
    List<Transition> transitions,
    HashSet<string> addedTransitions,
    Random random,
    int maxToAdd)
{
    int added = 0;
    for (int i = 0; i < states.Count - 1 && added < maxToAdd;
       i++)
    {
        var fromState = states[i].Id;
        var toState = states[i + 1].Id;
        var symbol = alphabet[random.Next(alphabet.Count)];

        var transition = new Transition
        {
            FromStateId = fromState,
            ToStateId = toState,
            Symbol = symbol
        };

        var key = GetTransitionKey(transition);
        if (!addedTransitions.Contains(key))
```

```
        {
            transitions.Add(transition);
            addedTransitions.Add(key);
            added++;
        }
    }

    return added;
}
```

Zdrojový kód 10: Zajištění grafové souvislosti

Statistické vlastnosti: Pro automat s parametry (n, m, k, r) (počet stavů, přechodů, velikost abecedy, podíl přijímajících) platí:

- Souvislost: 100 % (garantováno propojující cestou),
- Hustota přechodů: $\frac{m}{n \cdot k}$ (pro DKA shora omezena hodnotou 1),
- Procento přijímajících stavů: $r \pm \epsilon$ (kde ϵ závisí na zaokrouhlování),
- Využití abecedy: 100 % (garantováno).

4.3.6 Generování vstupních řetězců

Aplikace implementuje algoritmy pro automatizované generování zkušebních vstupů, jež jsou přijímány nebo zamítány daným automatem.

4.3.6.1 Generování přijímaného vstupu (BFS)

Algoritmus prohledává stavový prostor automatu pomocí BFS, kde každý uzel odpovídá dvojici (q, ℓ) – aktuální stav q a délka již přečteného vstupu ℓ. Velikost tohoto prostoru je $O(|Q| \cdot L_{\max})$, kde $L_{\max}$ je maximální povolená délka.

Vlastnost BFS: Stavy jsou prozkoumávány v pořadí rostoucí délky cesty. První nalezený přijímající stav tedy odpovídá *nejkratší* přijímané cestě.

```
private string? GenerateAcceptingStringViaBfs(
   AutomatonViewModel automaton,
    State startState, HashSet<int> acceptingStates, int
       maxLength)
{
    var queue = new Queue<(int StateId, string Path)>();
    var visited = new HashSet<(int, int)>();
    string? shortestAcceptingPath = null;

    queue.Enqueue((startState.Id, string.Empty));

    while (queue.Count > 0)
    {
        var (currentState, currentPath) = queue.Dequeue();
```

```
        if (currentPath.Length > maxLength)
            continue;

        if (acceptingStates.Contains(currentState))
        {
            if (currentPath.Length > 0)
            {
                return currentPath;
            }
            shortestAcceptingPath ??= currentPath;
        }

        var stateKey = (currentState, currentPath.Length);
        if (!visited.Add(stateKey))
            continue;

        EnqueueNextTransitions(automaton, queue, currentState,
            currentPath);
    }

    return shortestAcceptingPath;
}
```

Zdrojový kód 11: Hledání nejkratšího přijímaného řetězce pomocí BFS

Časová složitost činí $O(|Q| \cdot L_{\max} \cdot |\Sigma|)$.

4.3.6.2 Náhodná procházka (Random Walk)

Pro nedeterministické nebo strukturně složité automaty je přesnější algoritmus hledání náhodnou procházkou. uptavit **Prevence cyklů:** Implementace sleduje dvojice (q, ℓ) (stav, délka cesty). Pokud se stejná dvojice opakuje vícekrát (typicky 20 kroků bez nového objevu), je procházka ukončena.

4.3.6.3 Generování zamítaného vstupu

Algoritmus postupuje dvěma strategiemi:

1. **Systematické testování:** Postupně jsou generovány řetězce rostoucí délky a heuristicky testovány, zda by mohly být zamítnuty.

2. **Vzorové strategie (pro ZKA):** Speciální vzory porušující strukturu jazyka (např. nevyvážené závorky pro jazyk správně uzávorkovaných výrazů, nesprávné pořadí symbolů pro $a^n b^n$).

4.3.6.4 Souhrnná tabulka složitostí

Algoritmus	Nejlepší	Průměrný	Nejhorší	Paměť
Minimalizace DFA	$O(\|Q\| \cdot \|\Sigma\|)$	$O(\|Q\|^2 \cdot \|\Sigma\|)$	$O(\|Q\|^3 \cdot \|\Sigma\|)$	$O(\|Q\|^2)$
NFA $\rightarrow$ DFA	$O(\|Q\|^2 \cdot \|\Sigma\|)$	$O(\|Q\|^2 \cdot \|\Sigma\|)$	$O(2^{\|Q\|} \cdot \|\Sigma\|)$	$O(2^{\|Q\|})$
ε-NFA $\rightarrow$ NFA	$O(\|Q\|^2)$	$O(\|Q\|^2 \cdot \|\Sigma\|)$	$O(\|Q\|^2 \cdot \|\Sigma\| \cdot \|T\|)$	$O(\|Q\|^2)$
Thompson	$O(\|r\|)$	$O(\|r\|)$	$O(\|r\|)$	$O(\|r\|)$
Gen. vstupu (BFS)	$O(\|Q\|)$	$O(\|Q\| \cdot L)$	$O(\|Q\| \cdot L \cdot \|\Sigma\|)$	$O(\|Q\| \cdot L)$
Gen. vstupu (Random)	$O(L)$	$O(k \cdot L)$	$O(k \cdot L \cdot \|T\|)$	$O(L)$
Gen. automatu	$O(n \cdot m)$	$O(n \cdot m)$	$O(n \cdot m \cdot k)$	$O(n + m)$

Tabulka 1: Časové a prostorové složitosti implementovaných algoritmů. Symboly: $|Q|$ – počet stavů, $|\Sigma|$ – velikost abecedy, $|T|$ – počet přechodů, L – maximální délka slova, k – počet pokusů, $|r|$ – délka reg. výrazu, n, m – cílový počet stavů/přechodů.

4.4 Vizualizační systém a interaktivní plátno

Grafická reprezentace automatů je nezbytnou součástí edukační aplikace. Tato podkapitola popisuje architekturu a implementaci vizualizačního subsystému, který umožňuje dynamické vykreslování, editaci a animaci stavových diagramů přímo v prohlížeči uživatele.

4.4.1 Architektura vizualizačního systému

Vizualizační vrstva je postavena na modulární architektuře, která odděluje zodpovědnosti mezi několik specializovaných komponent. Celý systém je realizován pomocí moderního JavaScriptu (ES6 moduly) a využívá knihovnu Cytoscape.js jako grafový engine.

4.4.1.1 Klíčové komponenty

1. AutomatonCanvas (Hlavní fasáda)

Třída `AutomatonCanvas` slouží jako centrální vstupní bod a orchestrátor celého vizualizačního systému. Zapouzdřuje komplexitu nízkoúrovňového API Cytoscape.js a poskytuje vysokoúrovňové rozhraní pro načítání, zobrazení a manipulaci s automaty.

```
export class AutomatonCanvas {
    constructor(containerId, options = {}) {
        this.containerId = containerId;
        this.container = document.getElementById(containerId);

        this.options = {
            readOnly: options.readOnly ?? true,
            enablePanZoom: options.enablePanZoom ?? true,
            zoomOnWheel: options.zoomOnWheel ?? false,
            layoutName: options.layoutName ?? 'dagre',
```

```
            minZoom: options.minZoom ?? 0.3,
            maxZoom: options.maxZoom ?? 3,
            ...options
        };

        this.cy = null;
        this.automatonType = null;
        this.currentData = null;
        this.activeStateIds = [];
    }

    init() {
        this.cy = cytoscape({
            container: this.container,
            layout: { name: 'preset' },
            style: AutomatonRenderer.getStylesheet(this.
                automatonType),
            wheelSensitivity: this.options.wheelSensitivity,
            minZoom: this.options.minZoom,
            maxZoom: this.options.maxZoom,
            boxSelectionEnabled: false,
            autounselectify: this.options.readOnly,
            autoungrabify: false,
            userZoomingEnabled: !!(this.options.enablePanZoom
                &&
                                    this.options.zoomOnWheel),
            userPanningEnabled: this.options.enablePanZoom
        });
    }
}
```

Zdrojový kód 12: Inicializace vizualizačního plátna

2. AutomatonRenderer (Převod dat na graf)

Odpovídá za transformaci abstraktní datové reprezentace automatu (stavy a přechody v JSON) na grafovou strukturu srozumitelnou knihovně Cytoscape.js. Každému typu automatu (DKA, NKA, ε-NKA, ZKA) je přiřazen specializovaný renderer respektující jeho specifika.

```
static _renderStates(cy, states) {
    if (!states || states.length === 0) return;

    const nodes = states.map(state => ({
        group: 'nodes',
        data: {
            id: 'state-' + state.id,
            stateId: state.id,
            label: 'q' + state.id,
            isStart: state.isStart || false,
```

```
            isAccepting: state.isAccepting || false
        },
        classes: [
            state.isStart ? 'start' : '',
            state.isAccepting ? 'accepting' : ''
        ].filter(Boolean).join(' ')
    }));

    cy.add(nodes);
}
```

Zdrojový kód 13: Vykreslování stavů automatu

Přechody jsou před vykreslením seskupeny podle dvojice $(q_{\text{from}}, q_{\text{to}})$, aby bylo možné sloučit paralelní hrany vedoucí mezi stejnými stavy s různými symboly. Pro ZKA je formát značky hrany rozšířen o operace se zásobníkem:

```
if (isPDA) {
    const pdaLabels = transList.map(t => {
        const symbol = this._formatSymbol(t.symbol,
            allowEpsilon);
        const pop = this._formatSymbol(t.stackPop, true) || '
            eps';
        const push = t.stackPush || 'eps';
        return symbol + ', ' + pop + '/' + push;
    });
    label = pdaLabels.join('\n');
}
```

Zdrojový kód 14: Speciální vykreslování PDA přechodů

Výsledný popisek má tvar $a, X/\gamma$, kde a je vstupní symbol, X symbol odebíraný ze zásobníku a γ řetězec vkládaný na zásobník.

3. LayoutEngine (Automatické rozvržení)

Poskytuje adaptivní algoritmy pro automatické rozmístění uzlů na plátně tak, aby výsledný diagram byl vizuálně přehledný a snadno čitelný. Implementace podporuje šest různých layoutovacích strategií:

- **Dagre (Directed Acyclic Graph)**: Hierarchické rozvržení optimalizované pro orientované grafy. Využívá síťový Simplexový algoritmus pro minimalizaci křížení hran. Stavy jsou uspořádány do vrstev (ranks) zleva doprava.
- **Circle**: Uzly jsou umístěny na kružnici. Vhodné pro malé automaty (do 5 stavů).
- **Breadth-First**: Rozvržení dle prohledávání do šířky od počátečního stavu.

- **COSE (Compound Spring Embedder)**: Fyzikální simulace odpudivých a přitažlivých sil mezi uzly. Časově náročnější, ale vytváří esteticky vyrovnaná rozložení.

```
static _selectOptimalLayout(nodeCount, options) {
    if (nodeCount <= 5) {
        return 'circle';
    } else if (nodeCount <= 15) {
        return 'dagre';
    } else if (nodeCount <= 30) {
        return 'breadthfirst';
    } else {
        return 'cose';
    }
}
```

Zdrojový kód 15: Adaptivní výběr layoutu na základě velikosti grafu

Pro udržení konzistence mezi jednotlivými renderováními stejného automatu (např. po překreslení při změně režimu simulace) je implementována cache souřadnic uzlů. Tato cache je indexována pomocí tzv. *fingerprintu* – hašovacího otisku složeného z typu automatu a uspořádané sekvence identifikátorů stavů.

4. EditModeManager (Správa editačních režimů)
Systém rozlišuje tři základní režimy interakce:

- **View Mode (Prohlížecí režim):** Výchozí stav. Uživatel může prohlížet graf, přibližovat, oddalovat a posouvat viewport, ale nemůže měnit topologii automatu.
- **Edit Mode (Editační režim):** Umožňuje přidávání, mazání a úpravu stavů a přechodů.
- **Move Mode (Režim přesouvání):** Specifický režim, kde je možné ručně přesouvat uzly pro doladění vizuálního rozvržení.

Přepínání mezi režimy je zajištěno dynamickým (de)aktivováním event listenerů pomocí návrhového vzoru *Observer*.

5. StateEditor a TransitionEditor (Editační moduly)
Tyto komponenty implementují logiku pro interaktivní tvorbu a úpravu prvků grafu.

StateEditor reaguje na následující uživatelské akce:

- **Kliknutí na prázdné místo:** Vytvoření nového stavu na místě kliknutí.
- **Dvojklik na stav:** Přepnutí vlastnosti `isAccepting` (koncový/nekoncový).
- **Trojklik na stav:** Přepnutí vlastnosti `isStart` (počáteční/nepočáteční). Pouze jeden stav může být označen jako počáteční; změna automaticky odebere tuto vlastnost předchozímu počátečnímu stavu.

- **Kliknutí na stav:** Otevření kontextového menu s možnostmi úpravy vlastností stavu nebo jeho smazání.

```
_handleNodeClick(event) {
    const node = event.target;
    const tracker = this._clickTracker;

    if (tracker.nodeId !== node.id()) {
        tracker.nodeId = node.id();
        tracker.count = 1;
        tracker.timer = setTimeout(() => {
            tracker.count = 0;
            tracker.nodeId = null;
        }, tracker.clickDelay);
        return;
    }

    tracker.count++;
    clearTimeout(tracker.timer);

    if (tracker.count === 2) {
        tracker.timer = setTimeout(() => {
            this.toggleAcceptingState(node);
            tracker.count = 0;
            tracker.nodeId = null;
        }, tracker.clickDelay);
    } else if (tracker.count === 3) {
        this.toggleStartState(node);
        tracker.count = 0;
        tracker.nodeId = null;
    }
}
```

Zdrojový kód 16: Rozpoznávání vícenásobných kliknutí

TransitionEditor implementuje intuitivní dvoustupňový postup pro tvorbu přechodů:

1. Uživatel klikne na **zdrojový stav** – ten se vizuálně zvýrazní.
2. Uživatel klikne na **cílový stav** – otevře se modální dialog pro zadání symbolu přechodu (u ZKA navíc operací se zásobníkem).

Pro prevenci konfliktů s detekcí vícenásobných kliknutí na stav (dvojklik/trojklik) je výběr zdrojového stavu zpožděn o 320 ms. Pokud během této doby dojde k dalšímu kliknutí, je výběr zdrojového stavu potlačen.

6. TransitionDialog (Modální dialogové okno)
Specializovaná komponenta pro zadávání symbolů přechodů. Dialog je dynamicky generován přímo v DOM a není závislý na žádném externím CSS frameworku.

Klíčové vlastnosti:

- **Podpora ε-přechodů:** Dedikované tlačítko pro vložení epsilon symbolu.
- **Validace vstupu:** Kontrola, zda byl zadán alespoň jeden symbol (s výjimkou ε).
- **Rozšířené pole pro ZKA:** Pokud je automat typu ZKA, zobrazí se dodatečná pole `stackPop` a `stackPush`.
- **Escape handling:** Stisknutí klávesy Esc dialog zavře bez provedení změn.

7. ActionHistory (Historie akcí – Undo/Redo)
Implementace Command pattern pro podporu funkcionality zpět/vpřed (Undo/Redo). Každá editační akce (přidání stavu, smazání přechodu, změna vlastnosti) je zabalena do objektu obsahujícího dvojici funkcí `do()` a `undo()`.

```
this.actionHistory.recordAction({
    do: () => {
        this.cy.add({ group: 'nodes', data: {...}, position:
            {...} });
    },
    undo: () => {
        this.cy.getElementById(nodeId).remove();
    }
});
```

Zdrojový kód 17: Struktura záznamu akce

Historie je implementována jako zásobník s maximální kapacitou (výchozí 100 akcí). Při dosažení limitu jsou nejstarší záznamy zahozeny (FIFO).

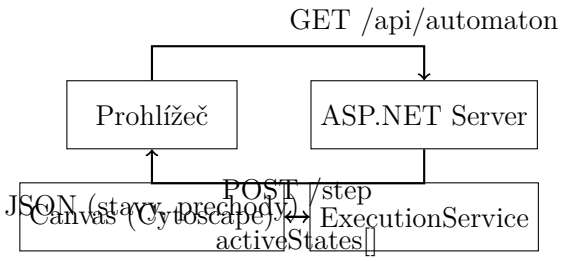
4.4.2 Interakce s backendem

Vizualizační vrstva je plně oddělena od serverové logiky. Komunikace probíhá výhradně prostřednictvím REST API endpointů, které vracejí data ve formátu JSON.

Typický datový tok:

1. **Načtení automatu:** HTTP GET požadavek na endpoint `/api/automaton/current` vrací aktuální stav automatu (stavy, přechody, typ).
2. **Transformace dat:** JSON odpověď je deserializována a předána metodě `AutomatonCanvas.loadAutomaton()`.
3. **Vykreslení:** Renderer převede data na grafovou reprezentaci a aplikuje layout.
4. **Uživatelská editace:** Změny provedené v edit módu jsou okamžitě zobrazeny na plátně. Pro perzistenci je nutné explicitní uložení pomocí tlačítka Save, které odešle HTTP POST na `/api/canvas/save`.

5. **Simulace:** Při spuštění simulace (krokovacího režimu) je posílán HTTP POST na `/api/execution/step` s aktuálním stavem. Server vrací nový stav včetně seznamu aktivních stavů, které jsou následně zvýrazněny na plátně voláním `highlight()`.



Obrázek 6: Schéma komunikace mezi frontendem a backendem během simulace

5 Zajištění kvality a testování

Kvalita dodaného softwaru není náhodou, ale výsledkem systematického a disciplinovaného přístupu k vývoji. V kontextu této diplomové práce, kde je implementována matematická logika transformací automatů, je robustní testovací strategie opravdu důležitá. V projektu je naimplementováno přes 1500 testovacích scénářů. (80% unit testů, 18% integračních testů a 2%end to end testů) Tato kapitola popisuje celkový přístup k zajištění kvality, metodiku tvorby testů a konkrétní testovací scénáře pokrývající kritické části aplikace.

5.1 Význam testování v moderním vývoji softwaru

Automatizované testování představuje jeden z klíčových pilířů moderního softwarového inženýrství. Zejména v akademickém a vědeckém kontextu, kde je kladen důraz na matematickou korektnost implementovaných algoritmů, slouží testy jako formální důkaz správnosti implementace.

5.1.1 Testy jako vývojářská pojistka

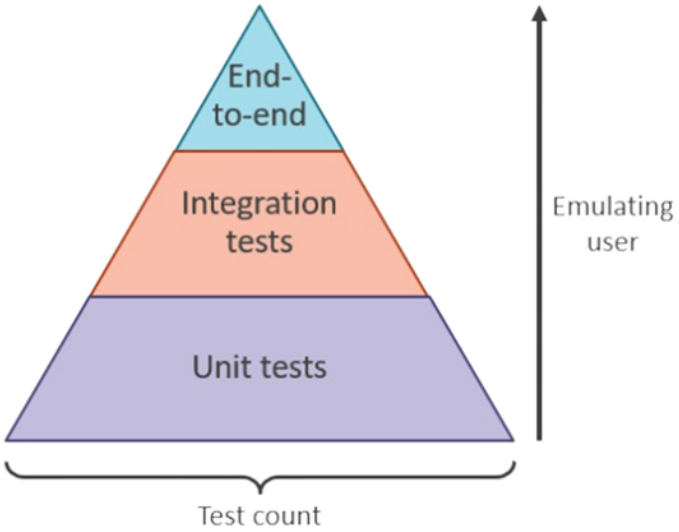
Na rozdíl od obchodních metrik (které primárně zajímají management a zákazníky), jsou automatizované testy především **nástrojem pro samotné vývojáře**. Tvoří bezpečnostní síť, která umožňuje:

- **Refaktoring bez obav:** Vývojář může s důvěrou přepisovat a optimalizovat kód, jelikož okamžitě zjistí, pokud něco rozbil.
- **Rychlou detekci regresí:** Při přidání nové funkcionality testy garantují, že existující chování zůstalo zachováno.

- **Živou dokumentaci:** Testy slouží jako čitelné příklady použití API a očekávaného chování systému.
- **Zvýšení sebedůvěry:** Programátor má jistotu, že kód dělá přesně to, co má.

5.2 Pyramida testů a testovací strategie

Pro efektivní a udržitelnou testovací strategii se v softwarovém inženýrství používá koncept *testovací pyramidy* [**?**]. Ta definuje ideální poměr různých typů testů v projektu.



Obrázek 7: Pyramida automatizovaných testů – čím níže ve vrstvě, tím vyšší počet testů

5.2.1 Jednotkové testy (Unit Tests) – Základ pyramidy

Tvoří nejpočetnější vrstvu testů. Ověřují správnost jednotlivých izolovaných komponent (tříd, metod, funkcí) nezávisle na zbytku systému.

Charakteristika:

- **Rychlost:** Extrémně rychlé (řádově milisekundy).
- **Izolace:** Žádné externí závislosti (databáze, síť, souborový systém).
- **Rozsah:** Testují jednu konkrétní funkcionalitu.
- **Počet:** Stovky až tisíce testů v běžném projektu.

Příklad z projektu: Testování matematických transformací automatů, jako je minimalizace nebo odstranění epsilon přechodů.

```
[Fact]
public void
   MinimizeDFA_WithRedundantStates_ShouldReduceStateCount()
{
    // Arrange: Priprava testovacich dat
    var dfa = new DFA();
    dfa.AddState(new State { Id = 1, IsStart = true });
    dfa.AddState(new State { Id = 2, IsAccepting = true });
    dfa.AddState(new State { Id = 3, IsAccepting = true });
    dfa.AddState(new State { Id = 4 });
    dfa.AddTransition(1, 2, 'a');
    dfa.AddTransition(1, 3, 'b');
    dfa.AddTransition(2, 4, 'a');
    dfa.AddTransition(3, 4, 'b');

    var minimizationService = new AutomatonConversionService()
       ;

    // Act: Provedeni minimalizace
    var minimized = minimizationService.MinimizeDFA(dfa);

    // Assert: Overeni vysledku
    minimized.States.Count.ShouldBeLessThan(dfa.States.Count);
    minimized.Execute("a").ShouldBeTrue();
    minimized.Execute("b").ShouldBeTrue();
}
```

Zdrojový kód 18: Ukázka jednotkového testu pro minimalizaci DFA

5.2.2 Integrační testy (Integration Tests) – Střední vrstva

Ověřují spolupráci více komponent dohromady a jejich interakci s reálnými externími systémy (databáze, API, souborový systém).

Charakteristika:

- **Rychlost:** Pomalejší než unit testy (řádově sekundy).
- **Integrace:** Zahrnují reálné závislosti.
- **Rozsah:** Testují celé scénáře použití.
- **Počet:** Desítky až stovky testů.

Příklad z projektu: Testování kompletního cyklu ukládání automatu do databáze a jeho následného načtení, včetně ověření persistence všech vlastností.

5.2.3 End-to-End testy (E2E) – Vrchol pyramidy

Simulují reálné chování uživatele v produkčním prostředí. Testují aplikaci jako celek od uživatelského rozhraní až po databázi.

Charakteristika:

- **Rychlost:** Nejpomalejší (řádově minuty).
- **Komplexita:** Křehké, náchylné k falešným poplachům.
- **Rozsah:** Kritické uživatelské cesty.
- **Počet:** Jednotky až desítky testů.

Příklad z projektu: Testování scénáře pomocí azure aspire.

5.3 Závislosti jako zdroj komplikací

Jeden z klíčových principů testovatelného kódu je minimalizace závislostí. Každá externí závislost (databáze, API, souborový systém) exponenciálně zvyšuje komplexitu testování.

Strategie minimalizace závislostí:

- **Dependency Injection:** Umožňuje nahradit reálné implementace mock objekty.
- **Testcontainers:** Eliminuje potřebu mockování databáze poskytnutím reálné instance.
- **Čistá architektura:** Doménová logika je zcela nezávislá na infrastruktuře.

5.4 Vlastnosti kvalitních testů

Nejen že testy musí být napsány, ale musí být napsány **správně**. Kvalitní test by měl splňovat následující vlastnosti:

- **Rychlost (Fast):** Test by měl běžet v řádu milisekund. Pomalé testy odvádějí pozornost vývojáře a snižují pravděpodobnost, že budou spouštěny často.
- **Opakovatelnost (Repeatable):** Test musí vždy vrátit stejný výsledek při stejném stavu systému. Náhodné selhávání (flaky tests) ničí důvěru v testovací sadu.
- **Automatizace (Automated):** Test nesmí vyžadovat manuální zásah. Kompletní běh testů by měl být spustitelný jediným příkazem.
- **Izolace (Isolated):** Testy nesmí záviset na pořadí spouštění ani na stavu zanechaném jiným testem. Každý test by měl být samostatnou jednotkou.
- **Čitelnost (Readable):** Test slouží jako živá dokumentace. Měl by být čitelný i pro programátora, který systém nenavrhoval.

5.5 Kontinuální integrace a automatizace

Všechny testy jsou integrovány do CI/CD pipeline (GitHub Actions). Při každém commitu do hlavní větve se automaticky spustí kompletní testovací sada a při neúspěchu informuje autora prostřednictvým emailu. Pro potřeby této diplomové práce to bohatě stačí, ovšem v praxi při práci v týmu je naprosto běžná praxe, že pokud neprojdou všechny testovací scénáře, tak systém nedovolí nahrát nový kód do hlavní vývojové větve. Na to navazují další automatizované scénáře, jako kontrola pokrytí kódu testy, dodržení kódovacích standardů a v neposlední řadě i příprava nasazení nového řešení a v některých případech i samotné nasazení do produkce.

5.6 Role testů při práci s umělou inteligencí

Vývoj softwaru s asistencí nástrojů umělé inteligence (jako je GitHub Copilot) přináší revoluci v produktivitě, avšak současně zvyšuje riziko zanesení chyb do kódu. AI modely jsou vynikající v generování syntakticky správného kódu, ale nemohou garantovat jeho sémantickou korektnost v kontextu konkrétní aplikace.

V tomto prostředí se automatizované testy stávají nepostradatelnou oporou. Fungují jako:

- **Validátor AI-generovaného kódu:** Okamžitě odhalí, pokud navržené řešení od AI nesplňuje specifikaci.
- **Zpětnovazební smyčka:** Umožňují iterativní zdokonalování AI návrhů – pokud test selže, lze AI požádat o opravu s konkrétním chybovým výpisem.
- **Prevence kognitivního přetížení:** Vývojář se nemusí soustředit na každý detail generovaného kódu; testy automaticky ověří správnost.

Navíc samotné psaní testů (hlavně jednotkových) je velmi rutinní záležitost, se kterou si moderní AI nástroje za dobrého použití hravě poradí. Tato diplomová práce byla částečně vyvíjena s využitím GitHub Copilot, přičemž robustní testovací pokrytí zajistilo, že kvalita výsledného softwaru zůstala na vysoké úrovni.

5.7 Závěr kapitoly

Systematický přístup k testování představuje páteř celého projektu. Robustní testovací pokrytí nejen garantuje korektnost implementovaných algoritmů, ale poskytuje vývojářům potřebnou jistotu pro experimentování a refaktoring. V kombinaci s nástroji jako Testcontainers a .NET Aspire bylo možné vytvořit realistické testovací prostředí, které věrně simuluje produkční chování aplikace.

Zkušenost z tohoto projektu jasně demonstruje, že investice do kvalitního testování se mnohonásobně vrátí ve formě snížené chybovosti, jednodušší údržby a celkově vyšší důvěry v dodávaný software – což jsou vlastnosti naprosto klíčové pro akademický a edukační nástroj, jakým tento simulátor konečných automatů bezpochyby je.

6 Uživatelská dokumentace

7 Testování a experimenty

8 Potenciální rozšíření a pokračování

Současná implementace simulátoru poskytuje solidní základ pro další vývoj. Tato kapitola identifikuje směry, kterými by se mohl projekt ubírat pro zvýšení jeho funkčnosti, edukačního potenciálu a výzkumné využitelnosti.

8.1 Funkční rozšíření

8.1.1 Turingovy stroje

Ačkoliv současná verze nepokrývá univerzální model výpočtu, přidání Turingových strojů (TM) by dokončilo pokrytí Chomského hierarchie.

8.1.2 Nedeterministické zásobníkové automaty

Současná verze podporuje pouze DPDA (deterministické PDA). Rozšíření na obecné nedeterministické PDA by umožnilo pokrýt všechny bezkontextové jazyky.

Výzvy:

- **Exponenciální větvení:** Nutnost vizualizace stromu výpočetních cest (podobně jako u NFA, ale s nekonečně hlubokými zásobníky).
- **Performance:** Prohledávání BFS/DFS s omezením hloubky (timeout mechanismus).
- **UI:** Interaktivní strom cest s možností rozkliknutí konkrétní větve.

Přínos: Kompletní pokrytí CFL (Context-Free Languages), umožnění demonstrace jazyka palindromů (klasický příklad nedeterminismu u PDA).

8.1.3 Formální gramatiky

Přidání podpory pro přímou práci s gramatikami (nikoliv pouze automaty).

Případové užití: Analýza syntaxe programovacích jazyků (ukázka parsování aritmetického výrazu).

8.1.4 Ekvivalence automatů

Implementace algoritmů pro testování, zda dva automaty přijímají stejný jazyk.

Přístupy:

1. **Pro DFA:** Konstrukce minimálních verzí obou automatů a jejich izomorfismus.

2. **Pro NFA:** Determinizace + minimalizace + izomorfismus.
3. **Obecně:** Konstrukce automatu pro symetrickou diferenci $L(A_1) \triangle L(A_2)$ a test prázdnosti.

8.2 Technická vylepšení

8.2.1 Progressive Web App (PWA)

Transformace na offline-first aplikaci.

Vlastnosti PWA:

- **Instalovatelnost:** Ikona na ploše, spuštění bez prohlížeče.
- **Offline režim:** Service Worker cachuje statické assety + IndexedDB pro data.
- **Push notifikace:** Upozornění na sdílené automaty, komentáře.

8.2.2 Mobilní responsivita

Adaptace UI pro tablety a smartphony.

8.2.3 Collaborative Real-Time Editing

Více uživatelů edituje tentýž automat současně (Google Docs-style).

8.3 AI asistence

8.3.1 Návrh automatu pomocí přirozeného jazyka

Použití Large Language Models (LLM) pro generování automatů z textových popisů.

Příklad:

> *User Input:* „Vytvoř automat, který přijímá binární čísla dělitelná třemi.“

8.4 Výzkumný potenciál

Kromě edukačního využití má aplikace potenciál pro výzkumné účely:

- **Sběr dat o učení:** Telemetrie (s consent) – Jak studenti skutečně navrhují automaty? Jaké dělají chyby? Kde tráví nejvíce času?
- **A/B testování UI:** Experimentování s různými vizualizačními přístupy (vliv barev, layoutu na porozumění).
- **Benchmark dataset:** Veřejná databáze uživatelsky vytvořených automatů (anonymizovaná) pro testování algoritmů.

Závěr

Tato diplomová práce se zabývala návrhem, implementací a evaluací interaktivního webového simulátoru pro výuku teorie formálních jazyků a automatů. Výsledkem je plně funkční aplikace, která kombinuje matematickou přesnost s moderním uživatelským rozhraním a robustní softwarovou architekturou.

Podařilo se mi spojit teoretické poznatky z oblasti formálních jazyků s praktickými dovednostmi v moderním softwarovém inženýrství. Výsledná aplikace není pouze akademickým cvičením, ale funkčním nástrojem s potencionálním dopadem na výuku.

Conclusions

Thesis conclusions in “English”.

A Návrh uživatelského rozhraní (HCI prezentace)

V elektronických datech (nebo na následujících stranách) je přiložena kompletní prezentace z předmětu Human Computer Interaction, mapující celý proces návrhu aplikace od definice cílové skupiny, přes HTA až po tvorbu prototypů.

B Testování při vývoji softwaru (Integrační testy pomocí testcontainers)

V elektronických datech je přiložena kompletní prezentace o testování, dokumentující využití Testcontainers, testovací pyramidy a best practices.

C Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

`text/`
: Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

`README.*`
: Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

`*`
: Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,
- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwarového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru `README.*`.

Literatura

[1] JANčAR, Petr. *Teoretická informatika: učební text*. Ostrava: Ediční středisko VŠB-TUO, 2007. 330 s. ISBN 978-80-248-1487-2.

[2] OSIčKA, Petr. *KMI/SLO - poznámky, ZS2021* [online]. Univerzita Palackého v Olomouci, 2022 [cit. 2026-03-06]. Dostupné z: https://phoenix.inf.upol.cz/~osicka/sloz/poznamky.pdf

[3] DEVOPEDIA. *Chomsky Hierarchy* [online]. 2026 [cit. 2026-02-20]. Dostupné z: https://devopedia.org/chomsky-hierarchy

[4] RODGER, Susan H., a kol. *JFLAP* [online]. Duke University, [cit. 2026-03-15]. Dostupné z: https://www.jflap.org/

[5] GEEKSFORGEEKS. *LIFO Principle in Stack* [online]. 2025 [cit. 2026-03-06]. Dostupné z: https://www.geeksforgeeks.org/dsa/lifo-principle-in-stack/

[6] ČERNÁ, Ivana; KřETÍNSKý, Mojmír; KUčERA, Antonín. *Formální jazyky a automaty I* [online]. Brno: Masarykova univerzita, 2006 [cit. 2026-03-06]. ISSN 1802-128X. Dostupné z: https://is.muni.cz/elportal/estud/fi/js06/ib005/Formalni_jazyky_a_automaty_I.pdf

[7] BALUN, Jiří. *Materiály ke cvičení z předmětu Formální jazyky* [online]. Univerzita Palackého v Olomouci, 2026 [cit. 2026-03-06]. Dostupné z: https://apollo.inf.upol.cz/~balun/fj/fj_cv_2026.pdf

[8] KLEENE, Stephen C. Representation of events in nerve nets and finite automata. In: *Automata studies*. Princeton, NJ: Princeton University Press, 1956.

[9] MITRA, Bivas. *Lecture notes: Thompson's Construction, Subset Construction* [online]. Indian Institute of Technology Kharagpur, 2013 [cit. 2026-03-08]. Dostupné z: https://cse.iitkgp.ac.in/~bivasm/notes/scribe/11CS10055.pdf

[10] MICROSOFT. *C# Documentation* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/csharp/

[11] MICROSOFT. *.NET Documentation* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/

[12] MICROSOFT. *ASP.NET Core Documentation* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/aspnet/core/

[13] MICROSOFT. *Introduction to Identity on ASP.NET Core* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity

[14] MICROSOFT. *Entity Framework Core* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/ef/core/

[15] MICROSOFT. *SQL Server Documentation* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/sql/sql-server/

[16] MICROSOFT. *.NET Aspire overview* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/aspire/get-started/aspire-overview

[17] W3C. *HTML5 Canvas* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://html.spec.whatwg.org/multipage/canvas.html

[18] W3C. *Cascading Style Sheets* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://www.w3.org/Style/CSS/

[19] BOOTSTRAP. *Bootstrap 5 Documentation* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://getbootstrap.com/docs/5.3/getting-started/introduction/

[20] MDN WEB DOCS. *JavaScript modules* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

[21] DOCKER INC. *Docker Documentation* [online]. 2026 [cit. 2026-03-09]. Dostupné z: https://docs.docker.com/

[22] XUNIT.NET. *xUnit.net Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://xunit.net/

[23] TESTCONTAINERS. *Testcontainers for .NET* [online]. [cit. 2026-03-15]. Dostupné z: https://dotnet.testcontainers.org/

[24] SHOULDLY. *Shouldly Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://docs.shouldly.org/

[25] OPENTELEMETRY. *OpenTelemetry Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://opentelemetry.io/docs/

[26] CYTOSCAPE CONSORTIUM. *Cytoscape.js Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://js.cytoscape.org/

[27] DAGRE. *Dagre: Directed graph layout for JavaScript* [online]. GitHub, [cit. 2026-03-15]. Dostupné z: https://github.com/dagrejs/dagre

[28] CYTOSCAPE CONSORTIUM. *Cytoscape.js-dagre Extension* [online]. GitHub, [cit. 2026-03-15]. Dostupné z: https://github.com/cytoscape/cytoscape.js-dagre

[29] MICROSOFT. *Visual Studio Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://learn.microsoft.com/cs-cz/visualstudio/

[30] DOCKER INC. *Docker Desktop* [online]. [cit. 2026-03-15]. Dostupné z: https://docs.docker.com/desktop/

[31] GITHUB INC. *GitHub Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://docs.github.com/

[32] GITHUB INC. *GitHub Copilot Documentation* [online]. [cit. 2026-03-15]. Dostupné z: https://docs.github.com/en/copilot

[33] KING, Brannon. *C# Best Practices: Dangers of Violating SOLID Principles in C#* [online]. MSDN Magazine, květen 2014 [cit. 2026-03-15]. Dostupné z: https://learn.microsoft.com/en-us/archive/msdn-magazine/2014/may/csharp-best-practices-dangers-of-violating-solid-principles-in-csha